Mathematics for AI

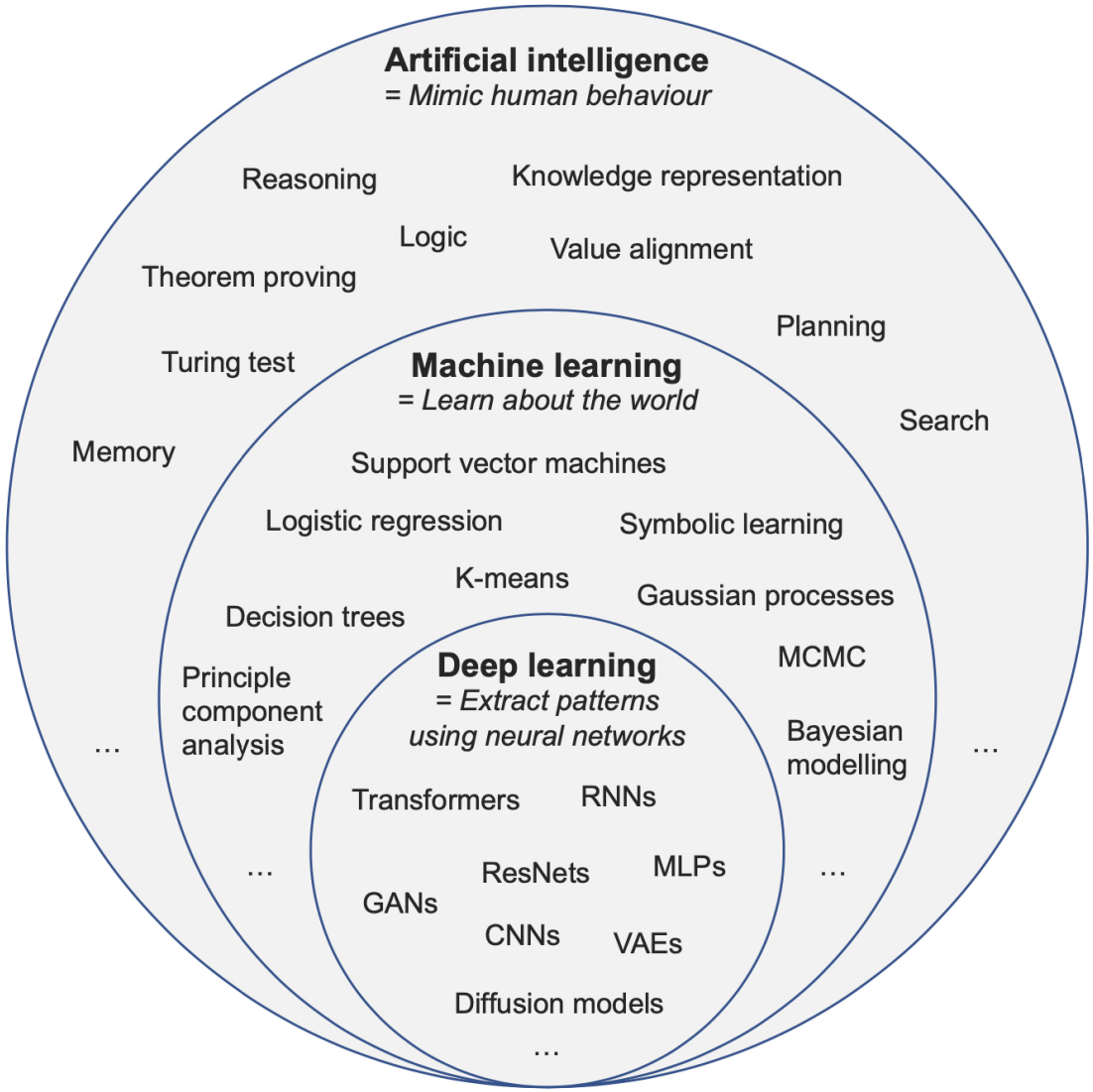
Supplementary Lecture Notes

Instructor

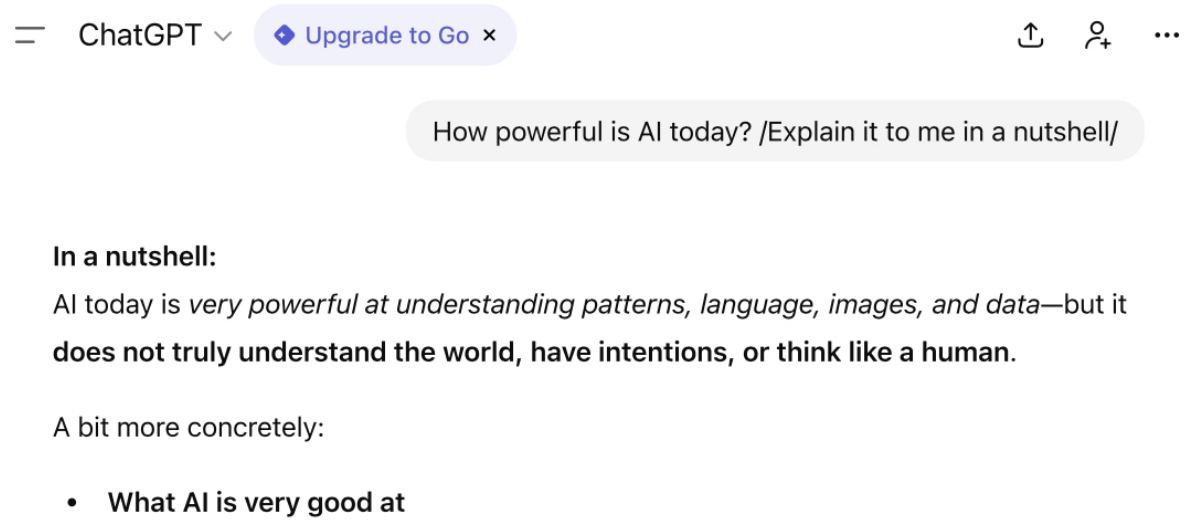
Imre Fekete

Motivation

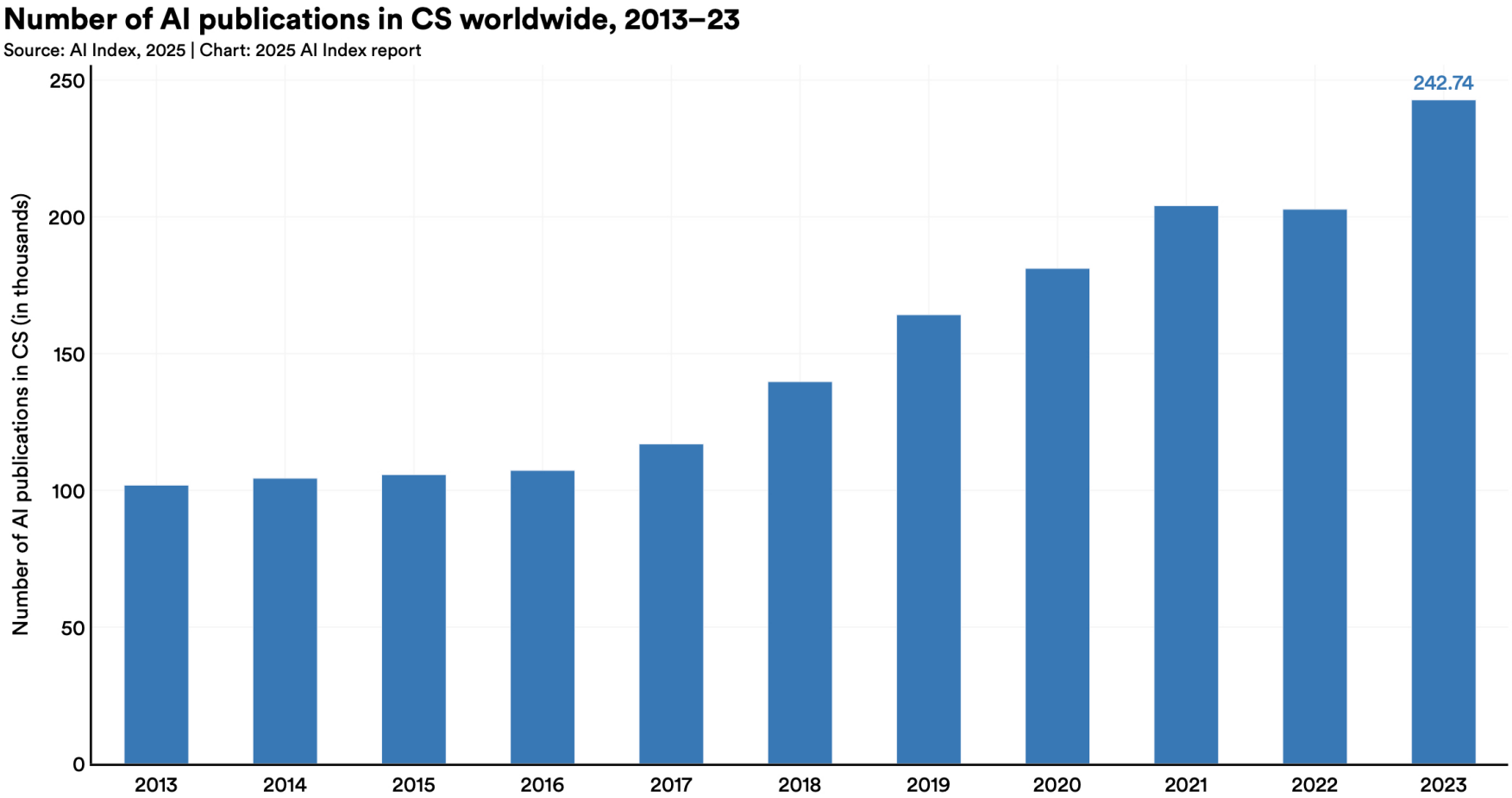
Big picture

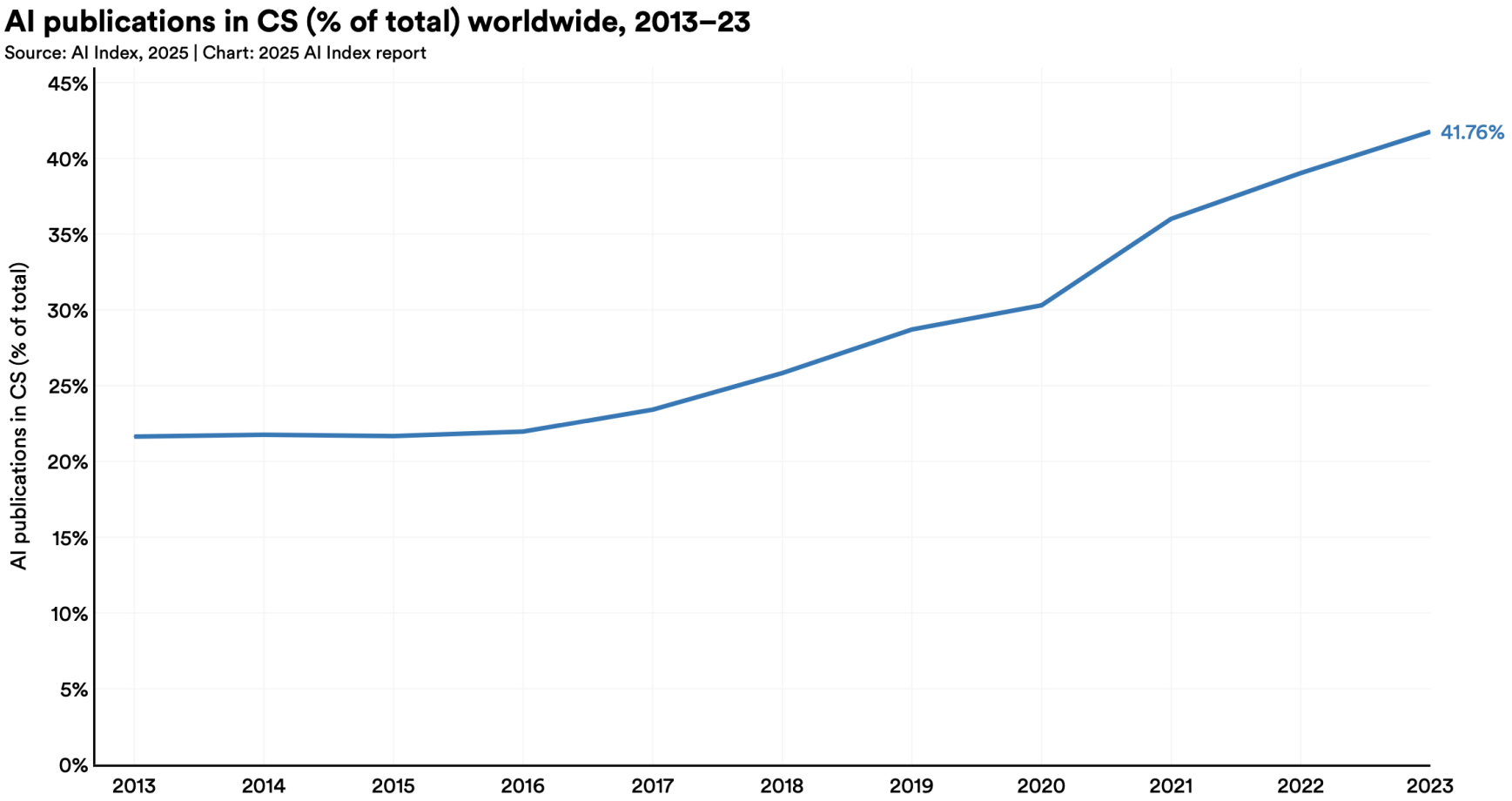


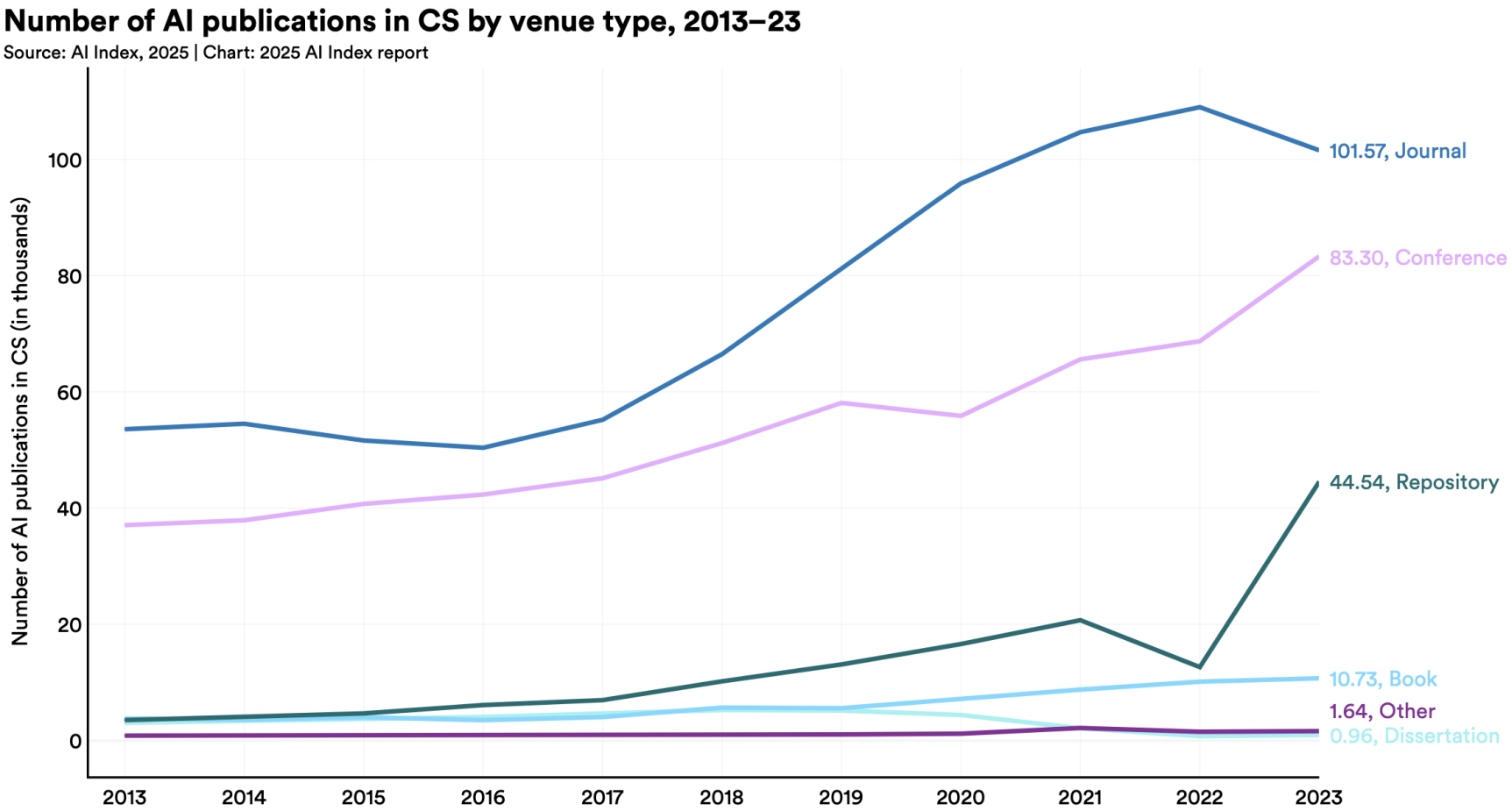
Source: 401-4656-21L AI in the Sciences and Engineering 2024



/Brown et al, Language Models are Few-Shot Learners, NeurIPS (2020)/


Number of AI publications in CS worldwide, 2013–23
Source: AI Index, 2025 | Chart: 2025 AI Index report
Number of AI publications in CS (in thousands)
250
200
150
100
50
0
242.74
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023


AI publications in CS (% of total) worldwide, 2013–23
Source: AI Index, 2025 | Chart: 2025 AI Index report
AI publications in CS (% of total)
45%
40%
35%
30%
25%
20%
15%
10%
5%
0%
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
41.76%


Number of AI publications in CS by venue type, 2013–23
Source: AI Index, 2025 | Chart: 2025 AI Index report
Number of AI publications in CS (in thousands)
100
80
60
40
20
0
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
101.57, Journal
83.30, Conference
44.54, Repository
10.73, Book
1.64, Other
0.96, Dissertation

© Nobel Prize Outreach. Photo: Nanaka Adachi

© Nobel Prize Outreach. Photo: Clément Morin

© Nobel Prize Outreach. Photo: Clément Morin

© Nobel Prize Outreach. Photo: Clément Morin

© Nobel Prize Outreach. Photo: Clément Morin

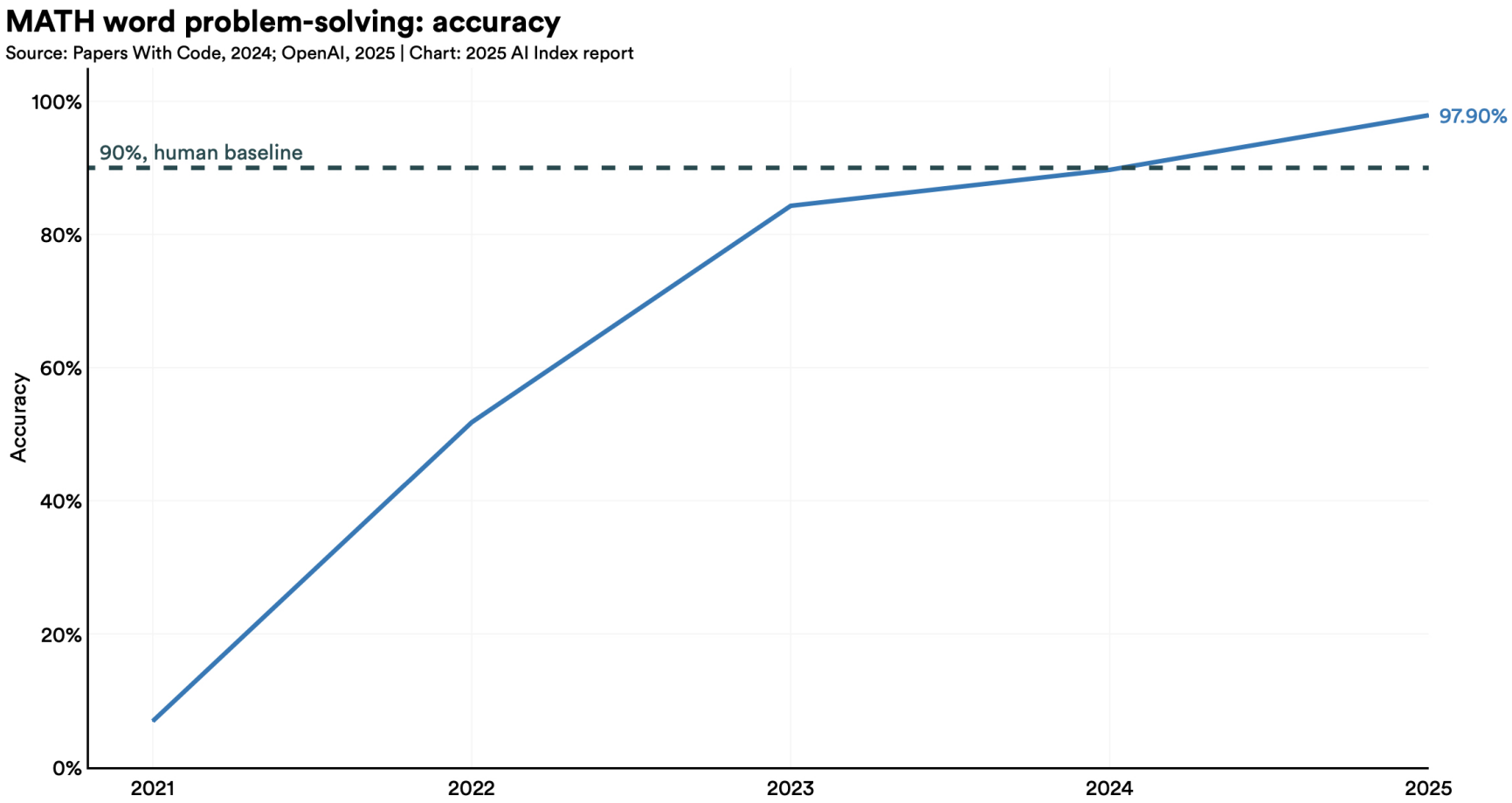
Physics

"for foundational discoveries and inventions that enable ML with artificial NNs"

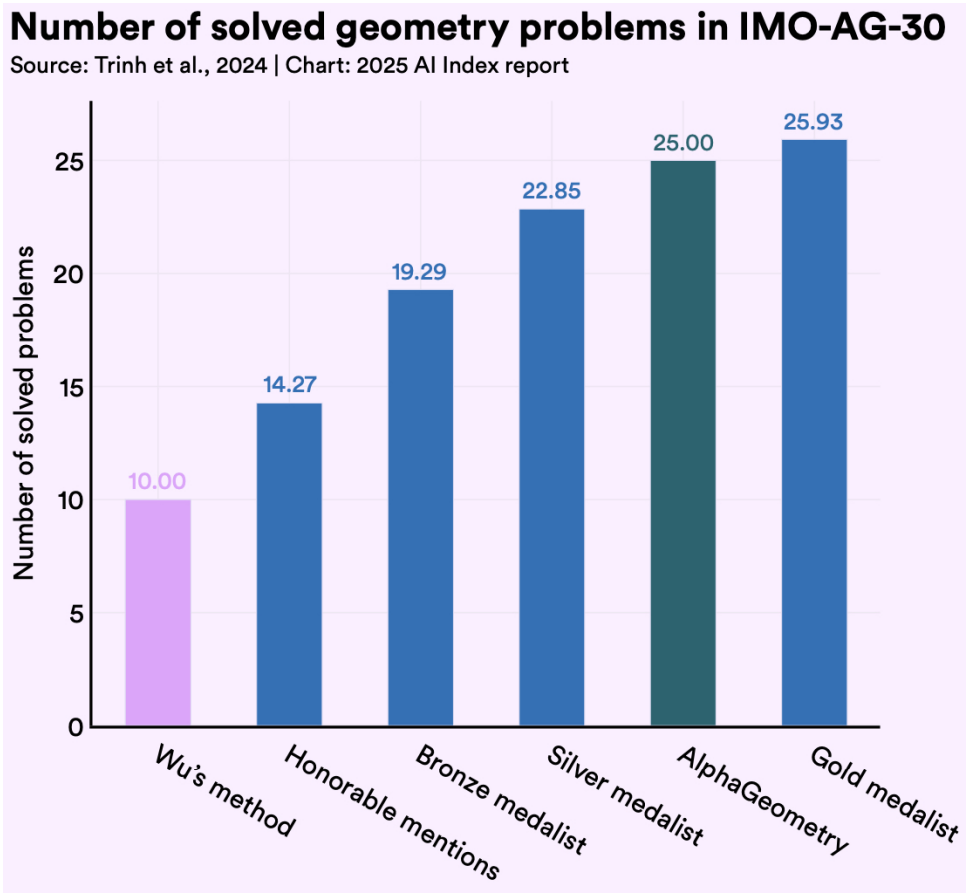
Chemistry

"for computational protein design" and "for protein structure prediction"

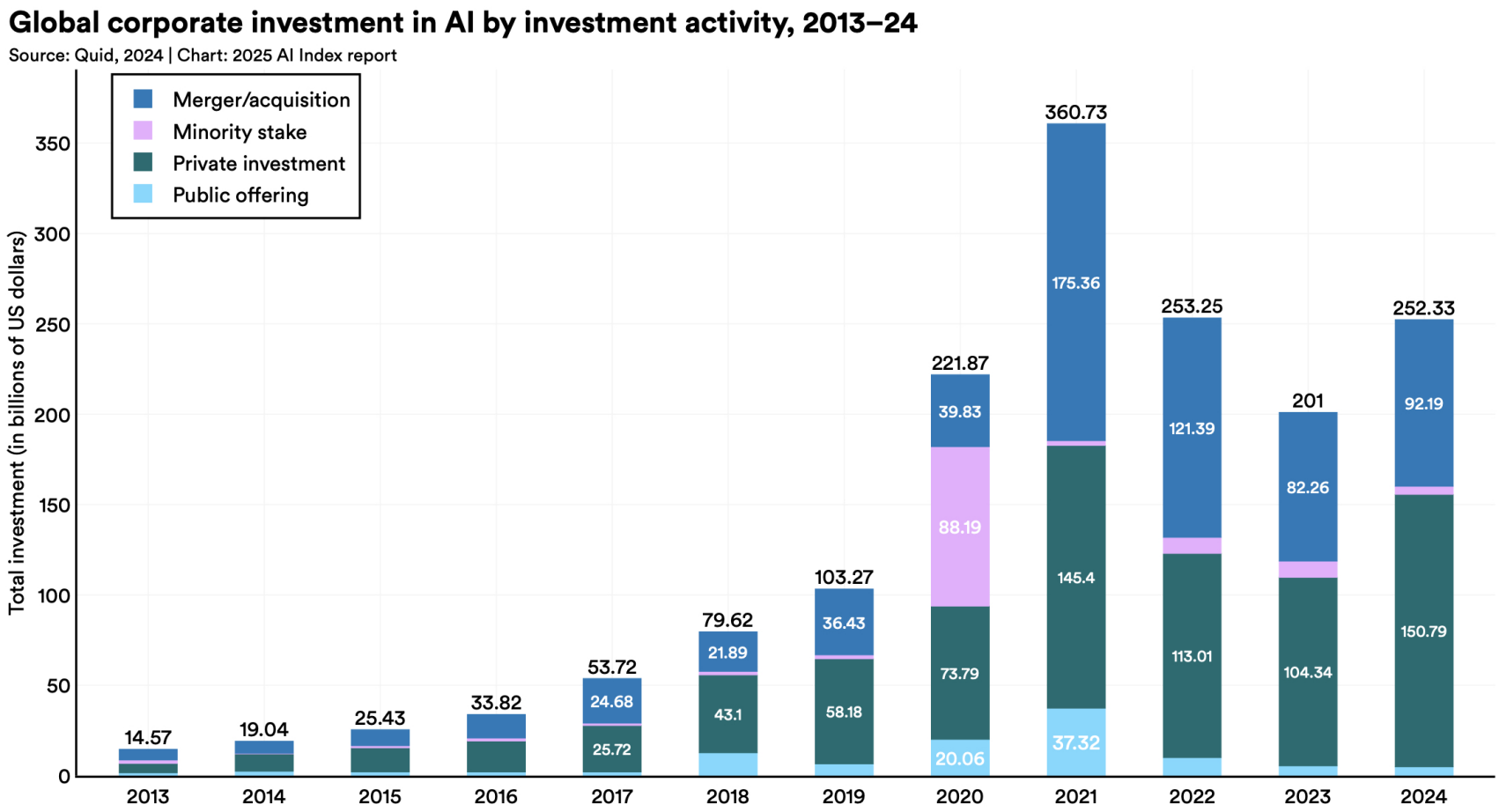
The Nobel Prize winners in 2024

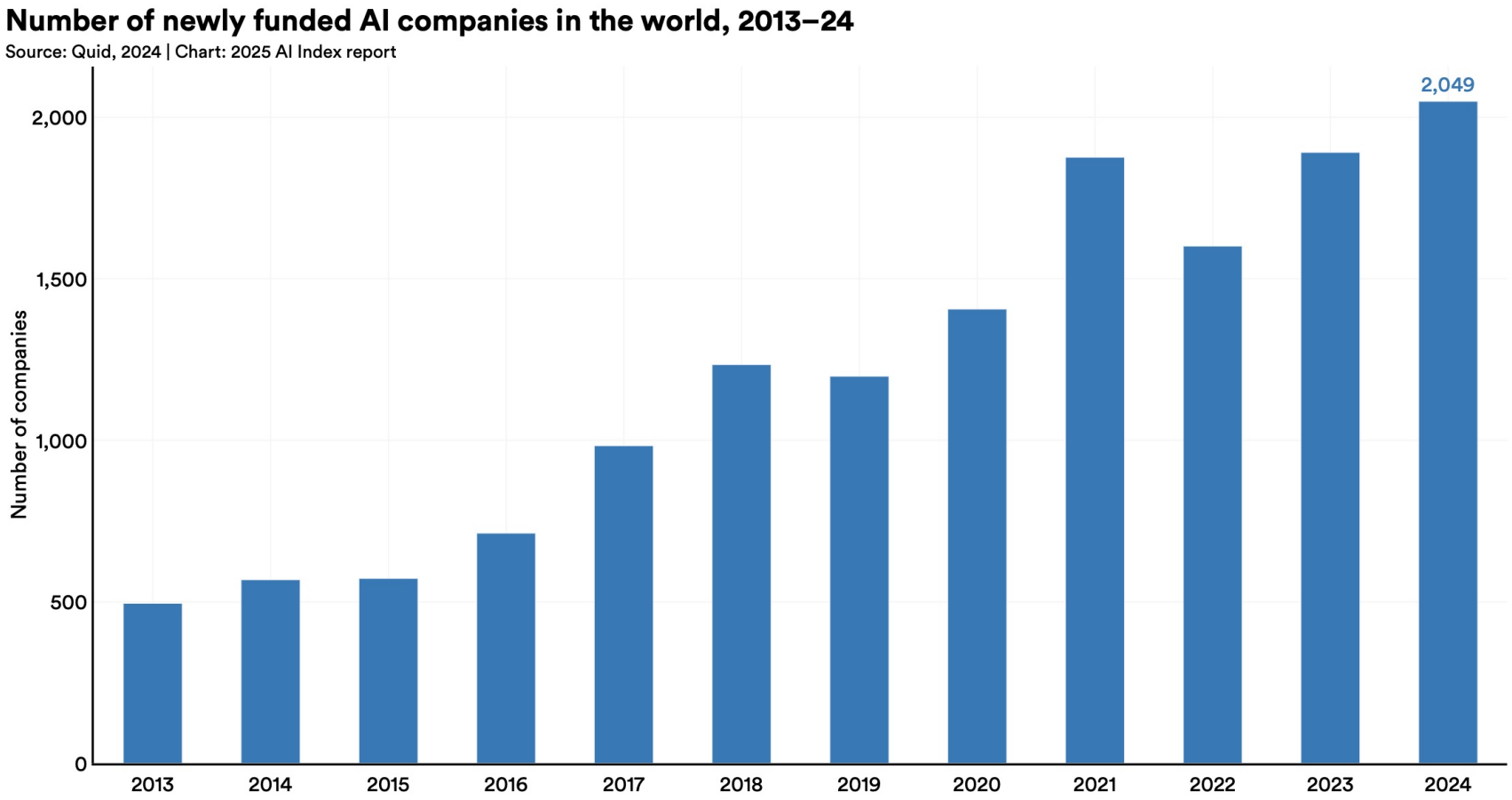


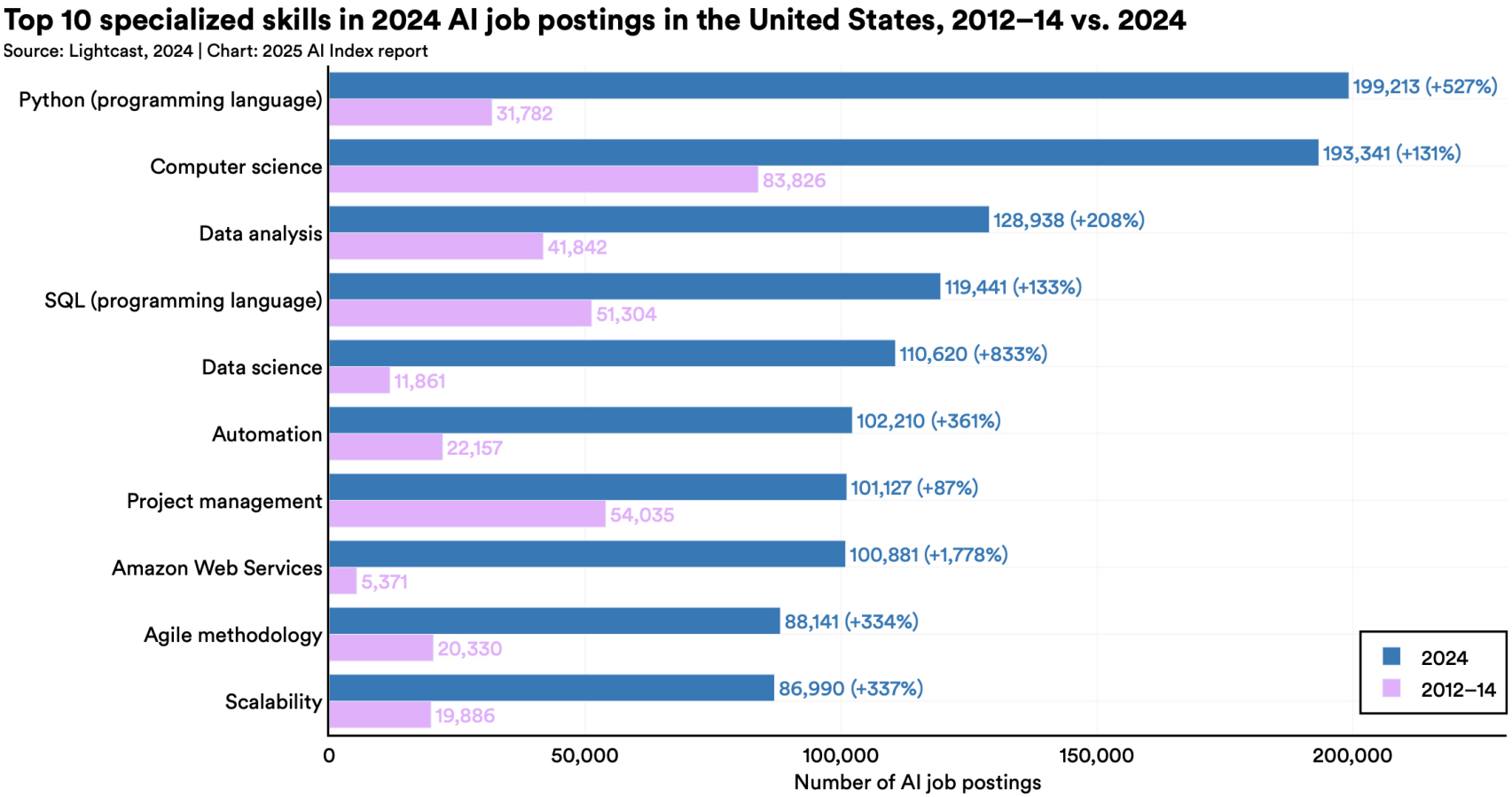
MATH is a dataset of 12,500 challenging, competition-level mathematics problems introduced by UC Berkeley and University of Chicago researchers in 2021. In January 2025, OpenAI's o3-mini (high) model was released and achieved the best performance.

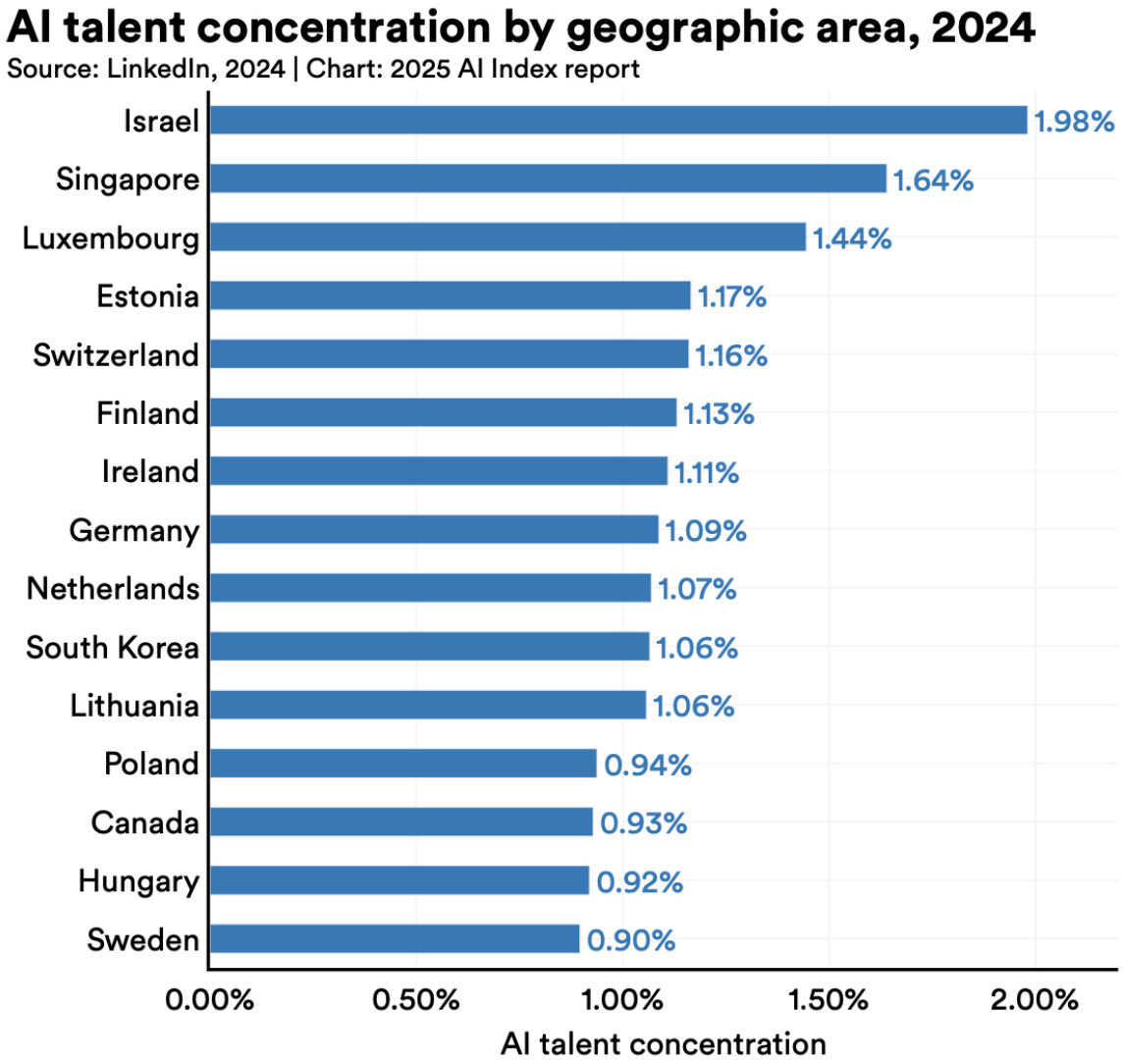


DeepMind employed its systems, AlphaProof (derived from AlphaZero, which was previously applied to chess, shogi, and Go) and AlphaGeometry 2, to solve four out of six problems in the 2024 International Mathematical Olympiad (IMO).


Global corporate investment in AI by investment activity, 2013–24
Source: Quid, 2024 | Chart: 2025 AI Index report
Merger/acquisition
Minority stake
Private investment
Public offering
Total investment (in billions of US dollars)
0
50
100
150
200
250
300
350
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
14.57
19.04
25.43
33.82
53.72
24.68
25.72
79.62
21.89
43.1
103.27
36.43
58.18
221.87
39.83
88.19
73.79
20.06
360.73
175.36
145.4
37.32
253.25
121.39
113.01
201
82.26
104.34
252.33
92.19
150.79

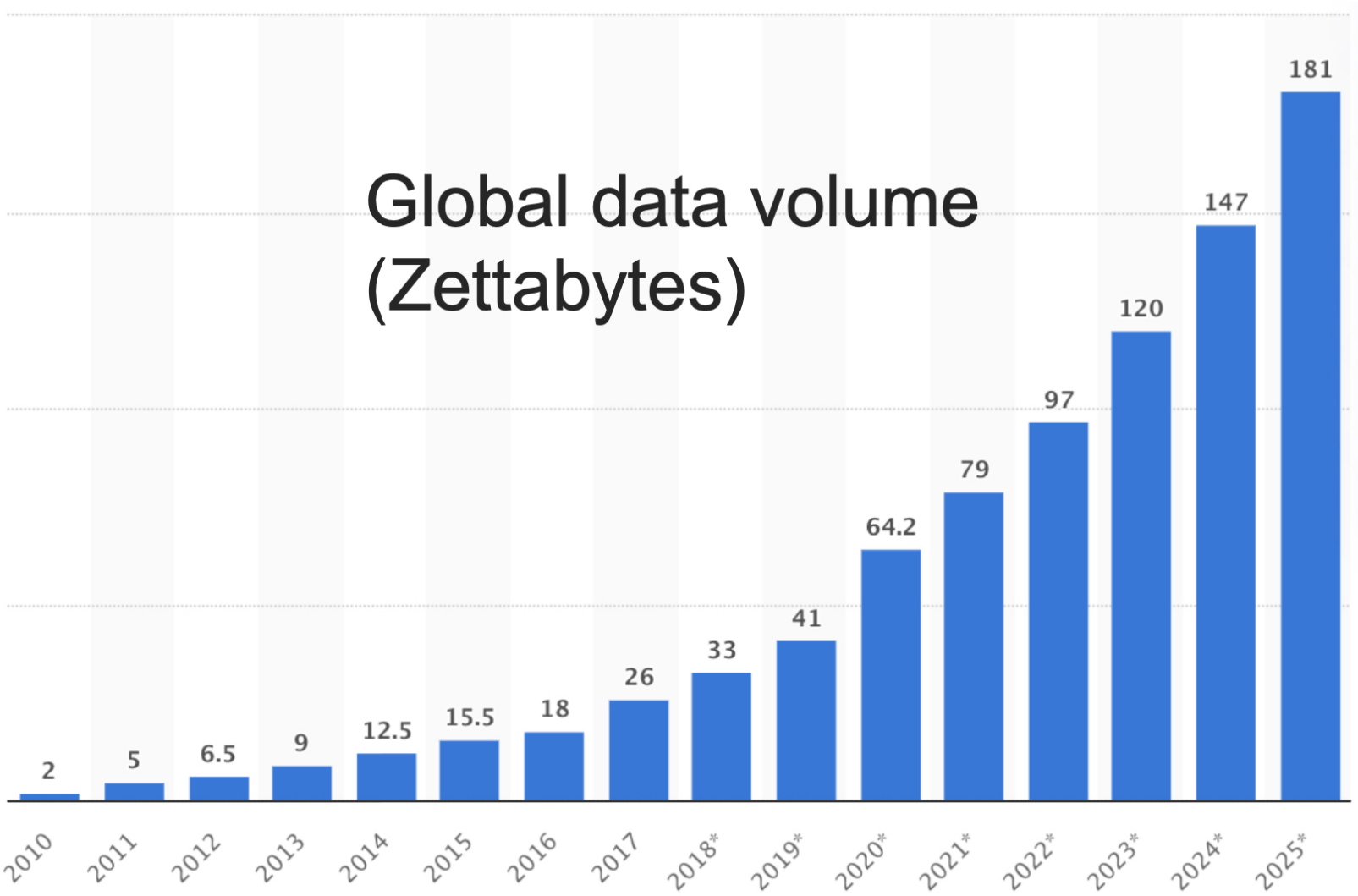

Number of newly funded AI companies in the world, 2013–24
Source: Quid, 2024 | Chart: 2025 AI Index report
Number of companies
2,000
1,500
1,000
500
0
2,049
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024


Top 10 specialized skills in 2024 AI job postings in the United States, 2012–14 vs. 2024
Source: Lightcast, 2024 | Chart: 2025 AI Index report
Python (programming language)
199,213 (+527%)
31,782
Computer science
193,341 (+131%)
83,826
Data analysis
128,938 (+208%)
41,842
SQL (programming language)
119,441 (+133%)
51,304
Data science
110,620 (+833%)
11,861
Automation
102,210 (+361%)
22,157
Project management
101,127 (+87%)
54,035
Amazon Web Services
100,881 (+1,778%)
5,371
Agile methodology
88,141 (+334%)
20,330
Scalability
86,990 (+337%)
19,886
2024
2012–14
0
50,000
100,000
150,000
200,000
Number of AI job postings


AI talent concentration by geographic area, 2024
Source: LinkedIn, 2024 | Chart: 2025 AI Index report
Israel 1.98%
Singapore 1.64%
Luxembourg 1.44%
Estonia 1.17%
Switzerland 1.16%
Finland 1.13%
Ireland 1.11%
Germany 1.09%
Netherlands 1.07%
South Korea 1.06%
Lithuania 1.06%
Poland 0.94%
Canada 0.93%
Hungary 0.92%
Sweden 0.90%
0.00% 0.50% 1.00% 1.50% 2.00%
AI talent concentration

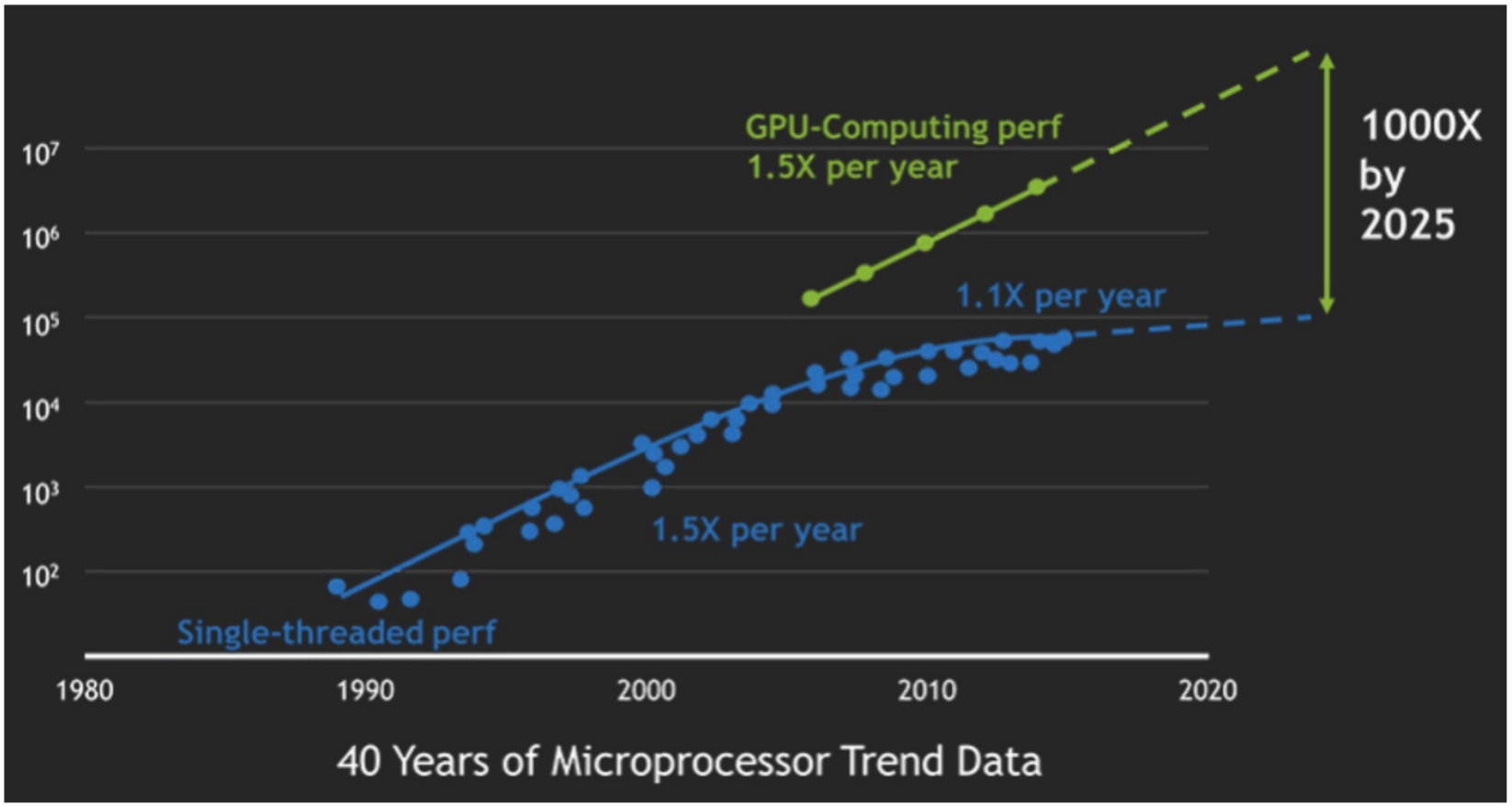
Why is DL so popular today?

Increasing amounts of data



Why is DL so popular today?

Hardware improvements

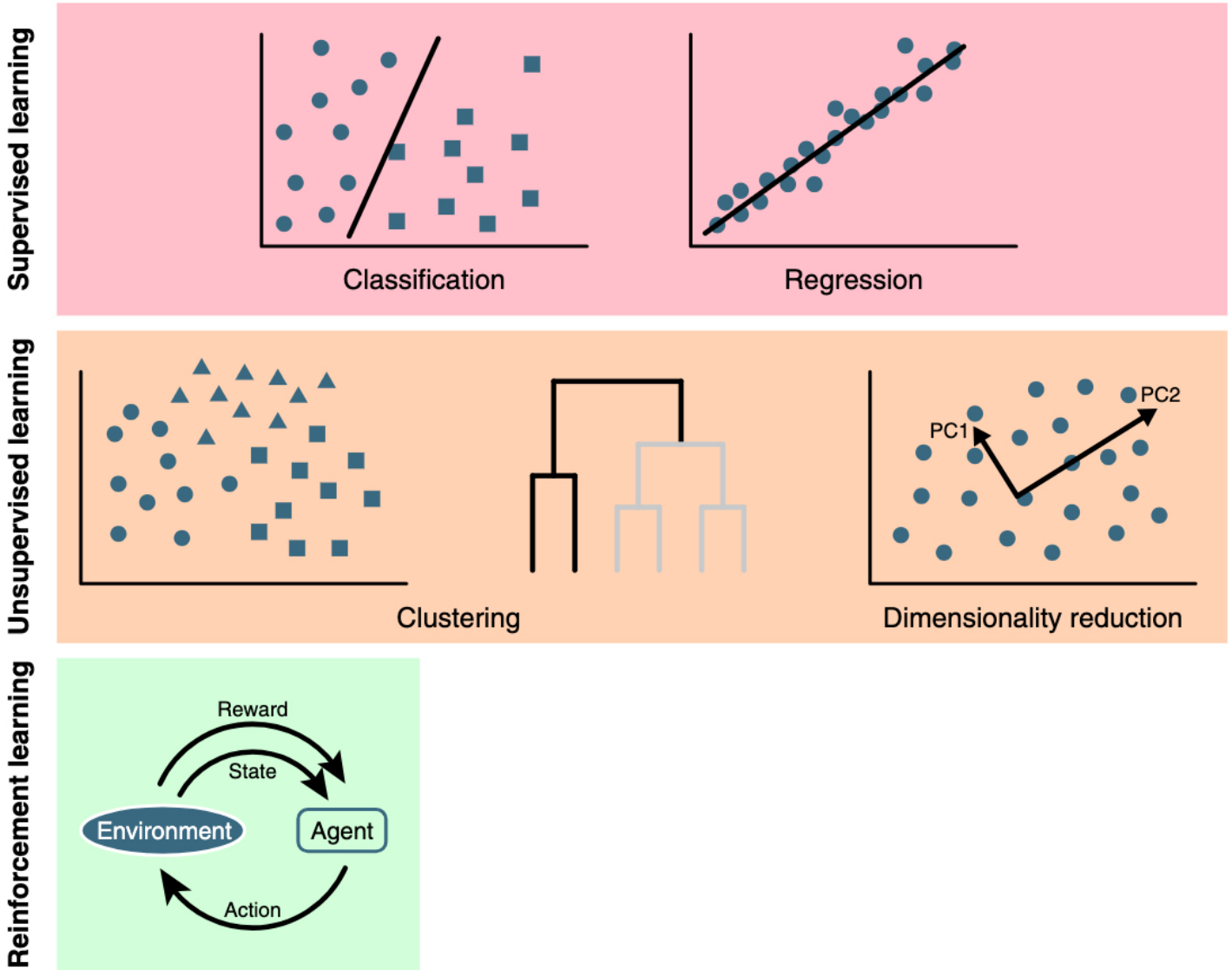


Source: NVIDIA

Why is DL so popular today?

Software improvements

Popular learning tasks

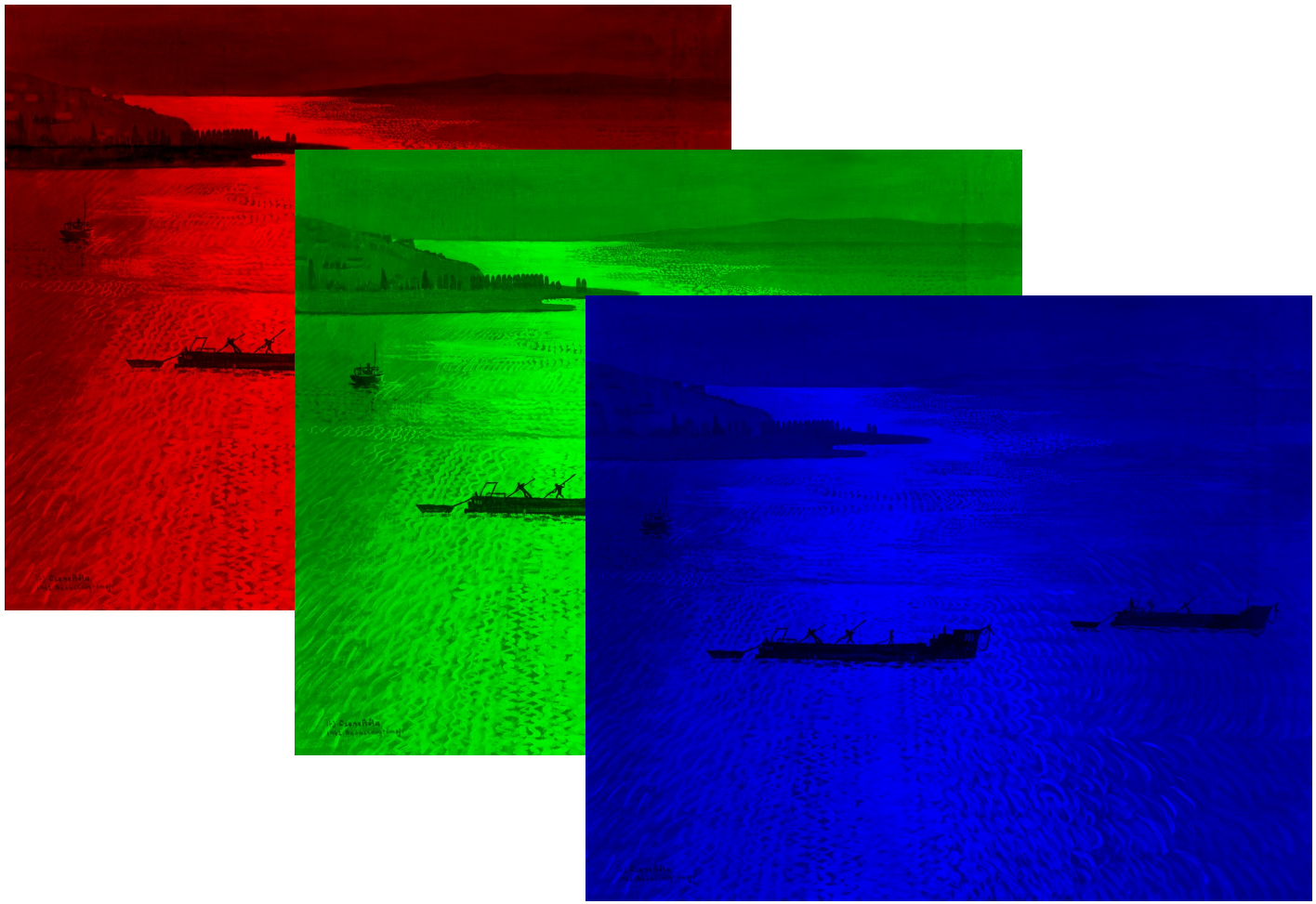


Source: DOI: 10.1007/s42994-023-00133-5

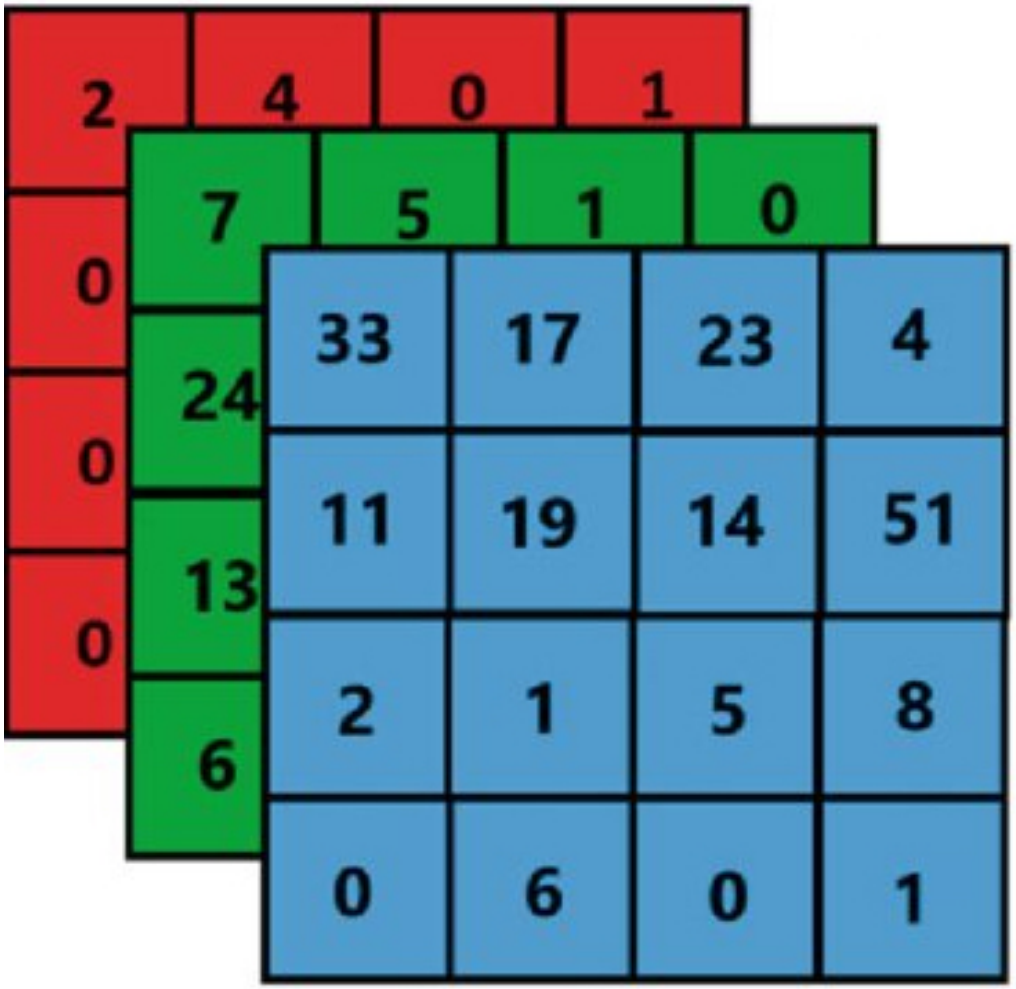
Example: RGB channels

Béla Czene: Sparkling lights on lake Balaton

Example: RGB channels



Example: RGB channels



Example: Tensor addition

X image

$$\underbrace{\begin{bmatrix} 3 & 5 & 4 & 2 \\ 7 & 1 & 6 & 0 \end{bmatrix}}_{\text{R}} \quad \underbrace{\begin{bmatrix} 1 & 2 & 0 & 6 \\ 3 & 4 & 0 & 2 \end{bmatrix}}_{\text{G}} \quad \underbrace{\begin{bmatrix} 0 & 1 & 3 & 1 \\ 0 & 2 & 5 & 3 \end{bmatrix}}_{\text{B}}$$

Example: Tensor addition

X image

$$\underbrace{\begin{bmatrix} 3 & 5 & 4 & 2 \\ 7 & 1 & 6 & 0 \end{bmatrix}}_{\text{R}} \qquad \underbrace{\begin{bmatrix} 1 & 2 & 0 & 6 \\ 3 & 4 & 0 & 2 \end{bmatrix}}_{\text{G}} \qquad \underbrace{\begin{bmatrix} 0 & 1 & 3 & 1 \\ 0 & 2 & 5 & 3 \end{bmatrix}}_{\text{B}}$$

Y image

$$\underbrace{\begin{bmatrix} 2 & 0 & 1 & 3 \\ 0 & 2 & 1 & 5 \end{bmatrix}}_{\text{R}} \qquad \underbrace{\begin{bmatrix} 3 & 5 & 4 & 0 \\ 1 & 2 & 3 & 1 \end{bmatrix}}_{\text{G}} \qquad \underbrace{\begin{bmatrix} 1 & 2 & 0 & 5 \\ 6 & 1 & 2 & 0 \end{bmatrix}}_{\text{B}}$$

Example: Tensor addition

$Z = X + Y$ image

$$\underbrace{\begin{bmatrix} 5 & 5 & 5 & 5 \\ 7 & 3 & 7 & 5 \end{bmatrix}}_{\text{R}} \qquad \underbrace{\begin{bmatrix} 4 & 7 & 4 & 6 \\ 4 & 6 & 3 & 3 \end{bmatrix}}_{\text{G}} \qquad \underbrace{\begin{bmatrix} 1 & 3 & 3 & 6 \\ 6 & 3 & 7 & 3 \end{bmatrix}}_{\text{B}}$$

Example: Tensor addition

$Z = X + Y$ image

$$\underbrace{\begin{bmatrix} 5 & 5 & 5 & 5 \\ 7 & 3 & 7 & 5 \end{bmatrix}}_{\text{R}} \qquad \underbrace{\begin{bmatrix} 4 & 7 & 4 & 6 \\ 4 & 6 & 3 & 3 \end{bmatrix}}_{\text{G}} \qquad \underbrace{\begin{bmatrix} 1 & 3 & 3 & 6 \\ 6 & 3 & 7 & 3 \end{bmatrix}}_{\text{B}}$$

$$Z \in C \otimes H \otimes W \cong Z_{\text{flat}} \in \mathbb{R}^{C \cdot H \cdot W}$$

Example: Tensor addition

$Z = X + Y$ image

$$\underbrace{\begin{bmatrix} 5 & 5 & 5 & 5 \\ 7 & 3 & 7 & 5 \end{bmatrix}}_{\text{R}} \qquad \underbrace{\begin{bmatrix} 4 & 7 & 4 & 6 \\ 4 & 6 & 3 & 3 \end{bmatrix}}_{\text{G}} \qquad \underbrace{\begin{bmatrix} 1 & 3 & 3 & 6 \\ 6 & 3 & 7 & 3 \end{bmatrix}}_{\text{B}}$$

$$Z \in C \otimes H \otimes W \cong Z_{\text{flat}} \in \mathbb{R}^{C \cdot H \cdot W}$$

$$Z = X + Y = \sum_{c=1}^{3} \sum_{i=1}^{2} \sum_{j=1}^{4} (X_{cij} + Y_{cij})\, e_c \otimes e_i \otimes e_j$$

$$Z_{\text{flat}} = [\underbrace{5, 5, 5, 5, 7, 3, 7, 5}_{R}, \underbrace{4, 7, 4, 6, 4, 6, 3, 3}_{G}, \underbrace{1, 3, 3, 6, 6, 3, 7, 3}_{B}]^T$$

Example: Tensor addition


```
import torch

X = torch.tensor([[[3,5,4,2],[7,1,6,0]], [[1,2,0,6],[3,4,0,2]],
                  [[0,1,3,1],[0,2,5,3]]], dtype=torch.float32)
Y = torch.tensor([[[2,0,1,3],[0,2,1,5]], [[3,5,4,0],[1,2,3,1]],
                  [[1,2,0,5],[6,1,2,0]]], dtype=torch.float32)

Z = X + Y

print(Z)
```

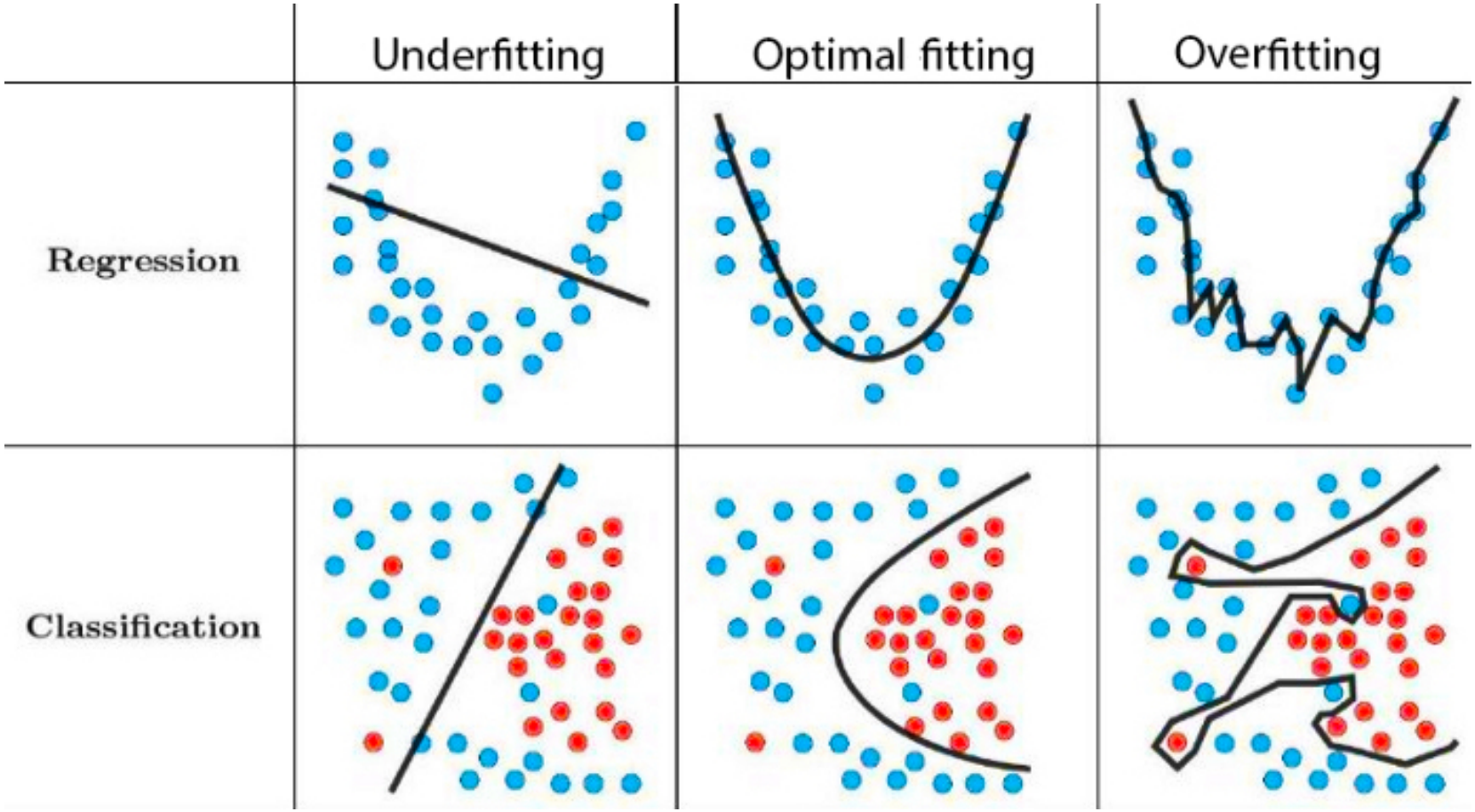
```
tensor([[[5., 5., 5., 5.],
         [7., 3., 7., 5.]],

        [[4., 7., 4., 6.],
         [4., 6., 3., 3.]],

        [[1., 3., 3., 6.],
         [6., 3., 7., 3.]]])
```

Example: Tensor addition

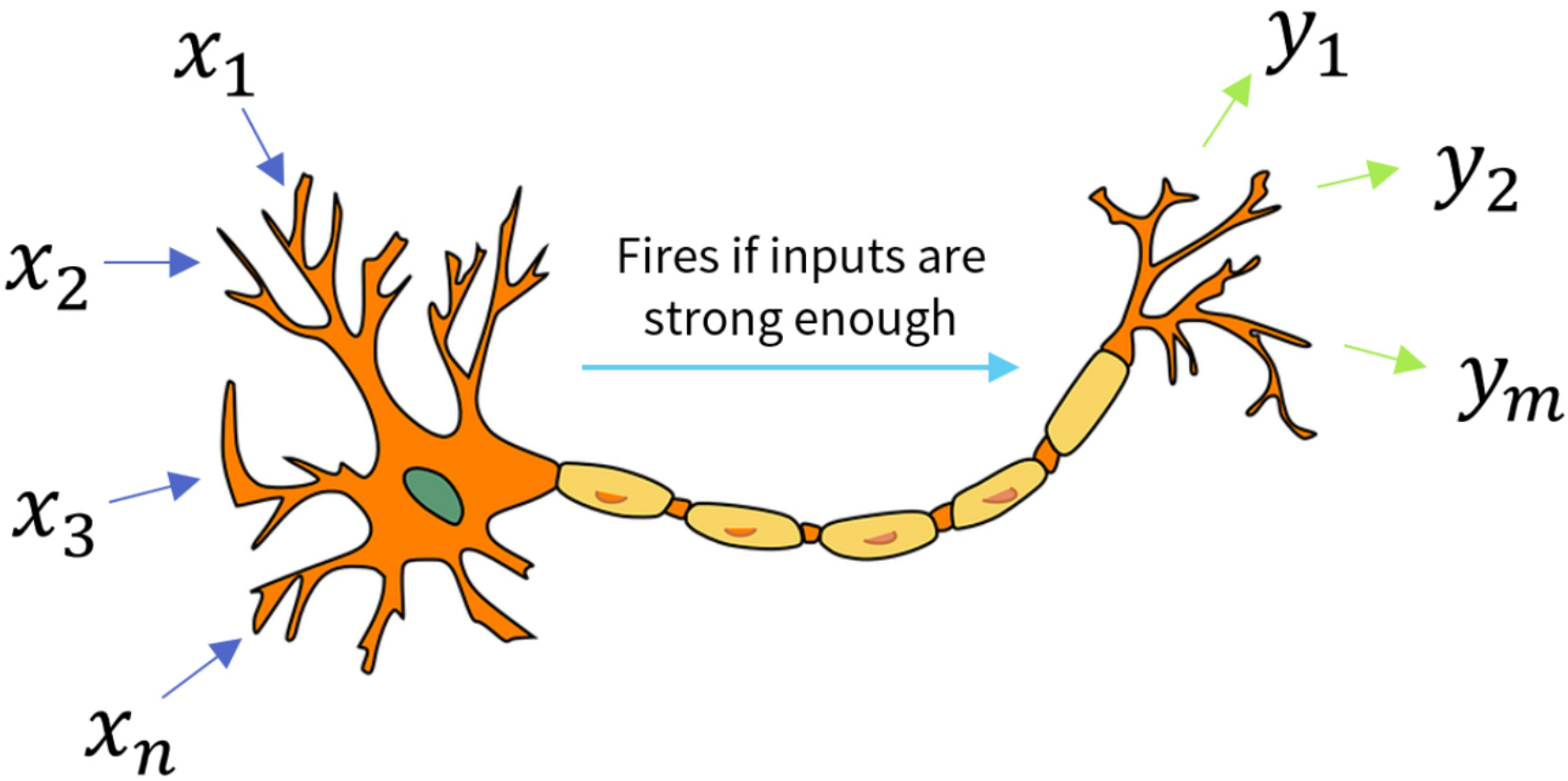
```
import tensorflow as tf

X = tf.constant([[[5,3,6,7],[2,1,0,4]], [[4,2,1,0],[5,3,7,2]],
                 [[1,0,2,3],[4,6,5,1]]], dtype=tf.float32)
Y = tf.constant([[[1,1,1,1],[1,1,1,1]], [[0,1,0,1],[1,0,1,0]],
                 [[2,0,1,0],[0,1,0,1]]], dtype=tf.float32)

Z = X + Y

print(Z)
```

```
tf.Tensor(
[[[6. 4. 7. 8.]
  [3. 2. 1. 5.]]

 [[4. 3. 1. 1.]
  [6. 3. 8. 2.]]

 [[3. 0. 3. 3.]
  [4. 7. 5. 2.]]], shape=(3, 2, 4), dtype=float32)
```

Underfitting and overfitting

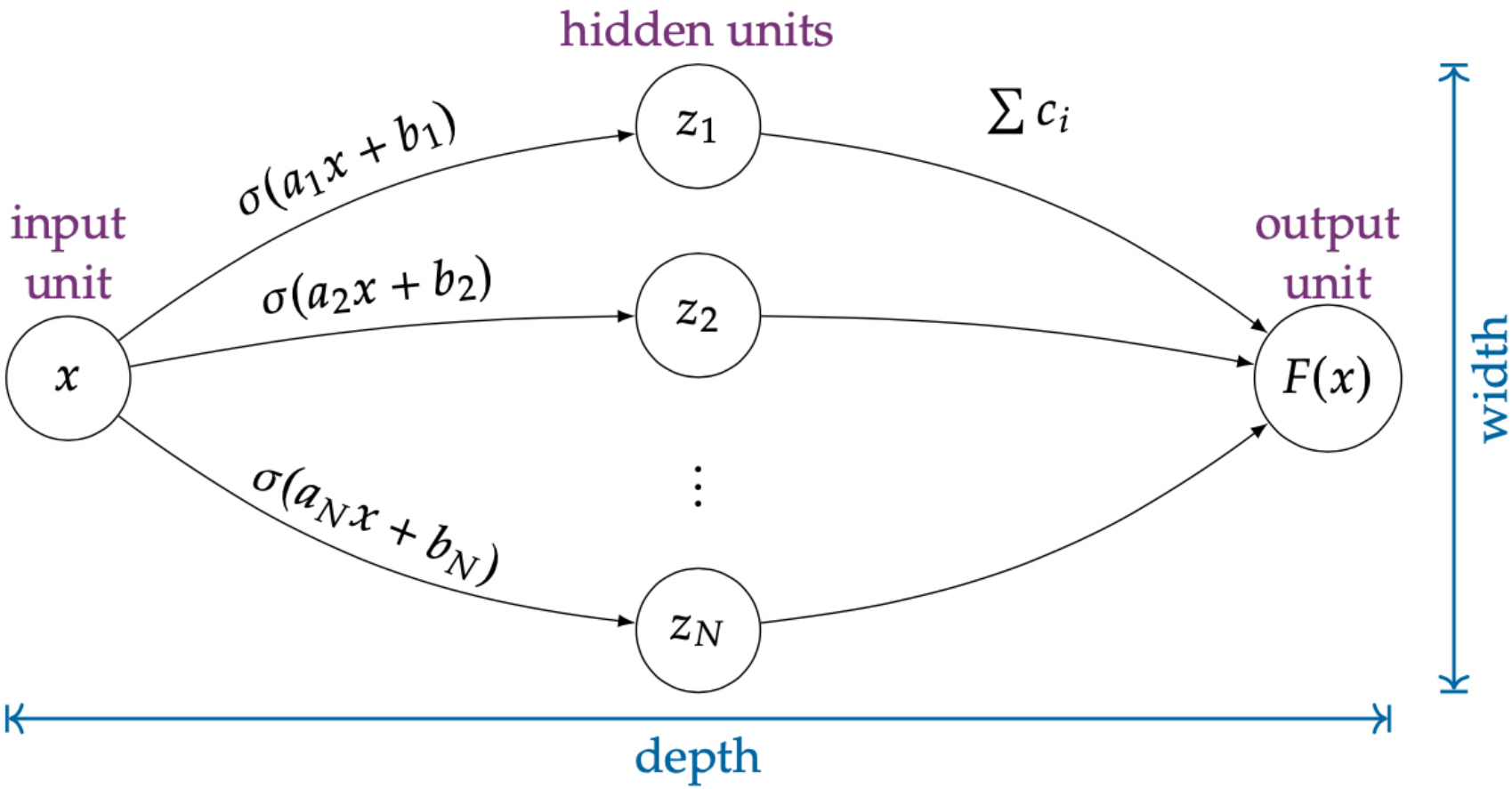


Biological inspiration of artificial NNs

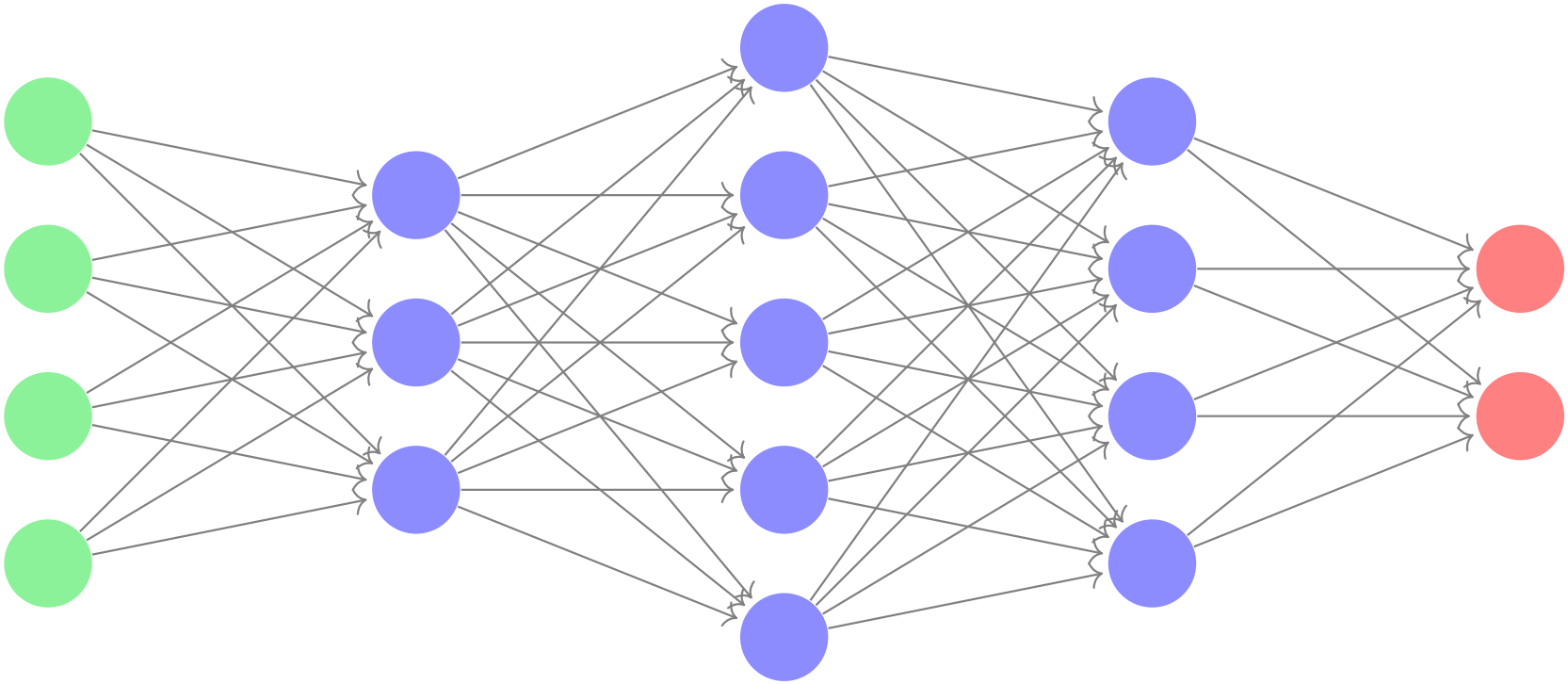


Dendrites receive signals, and once a threshold is reached, the neuron fires through its axon and synapses /Source: arXiv:2403.04807v1/

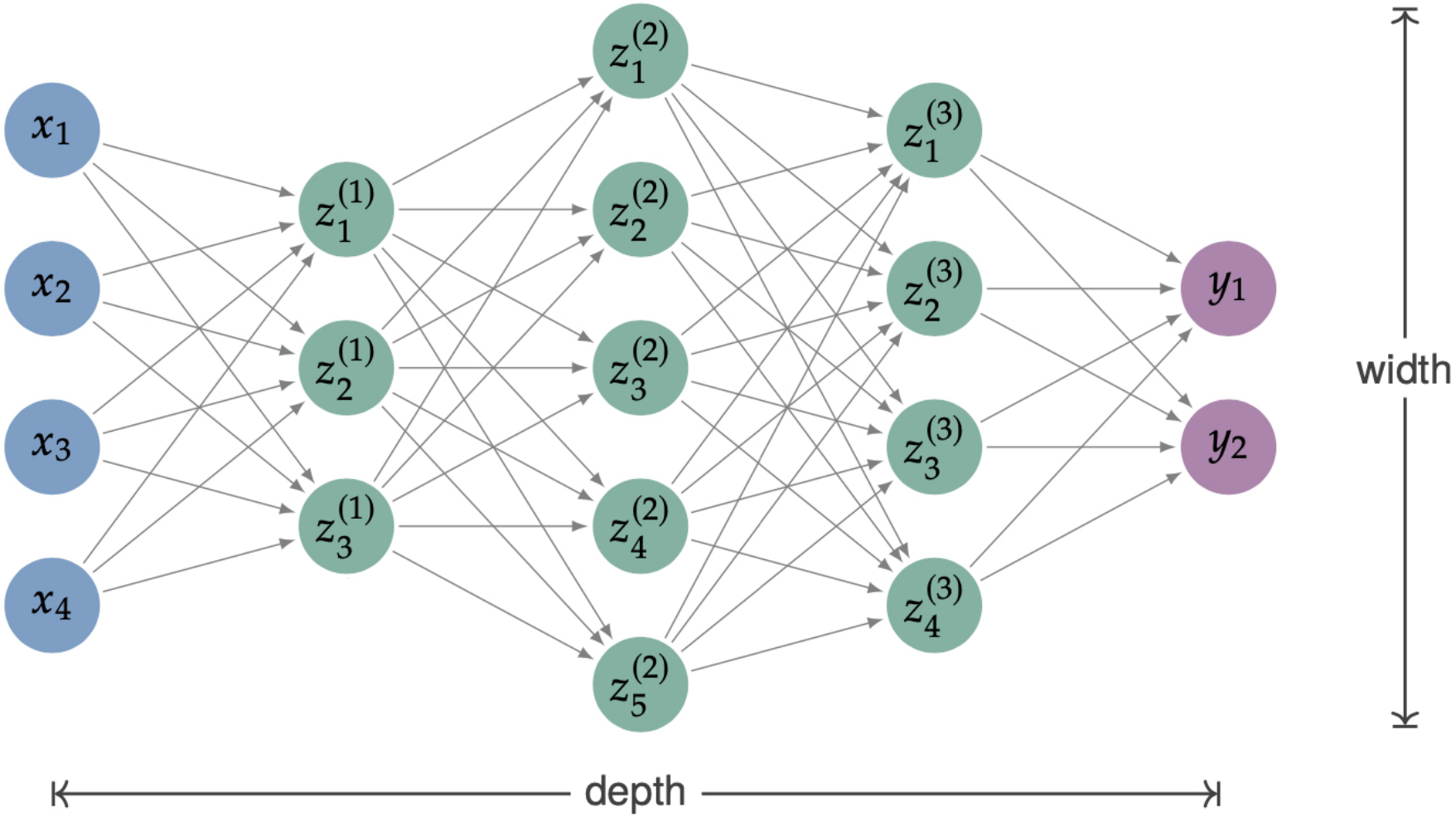
Shallow NN


hidden units
input unit
output unit
x
z_1
z_2
z_N
$\sigma(a_1x + b_1)$
$\sigma(a_2x + b_2)$
$\sigma(a_Nx + b_N)$
$\sum c_i$
$F(x)$
width
depth

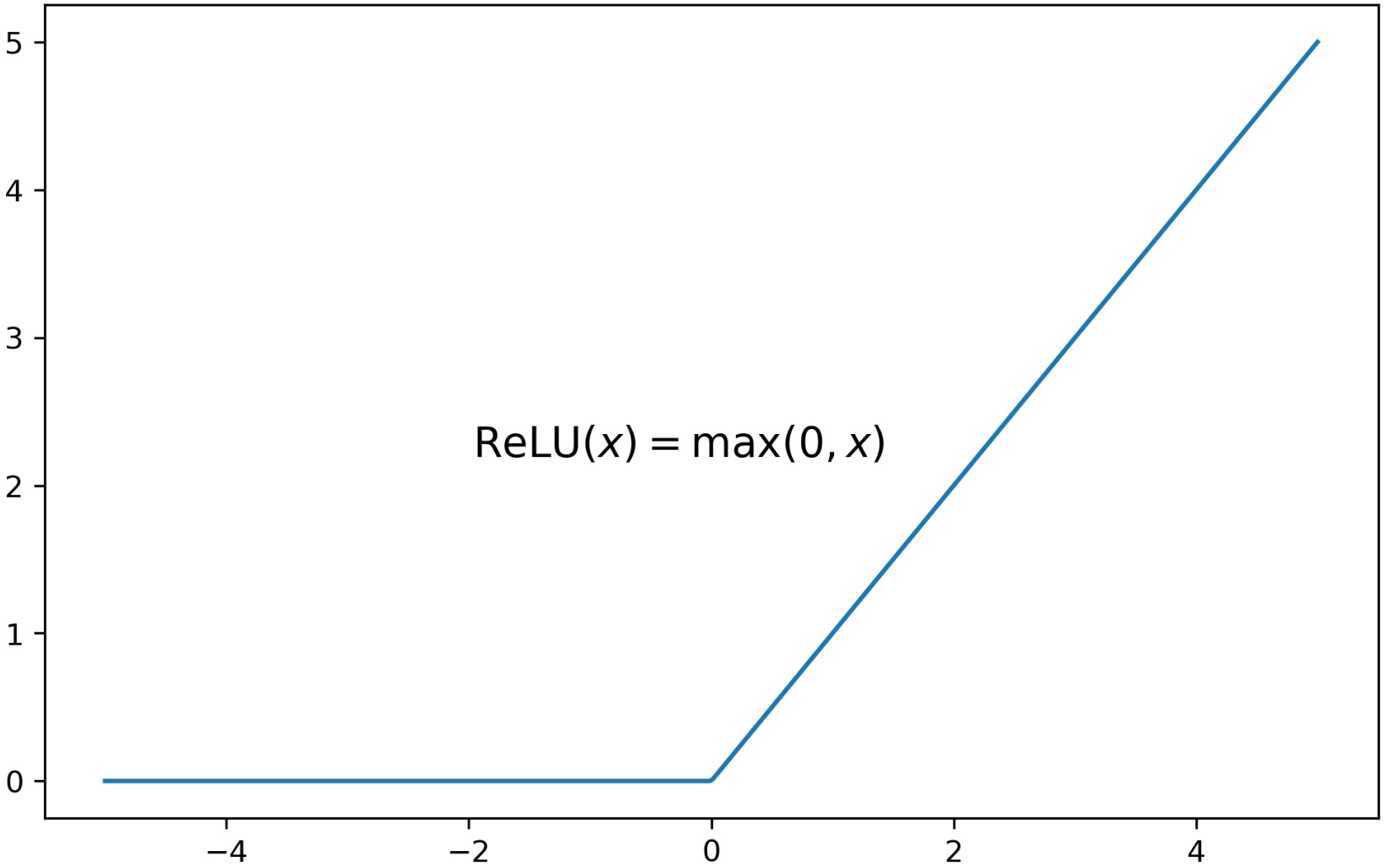
Feedforward NN



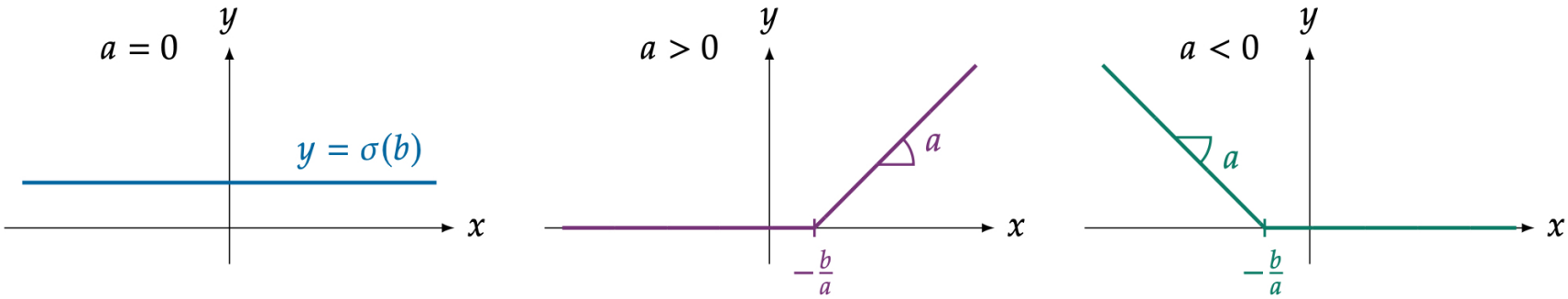
Feedforward NN



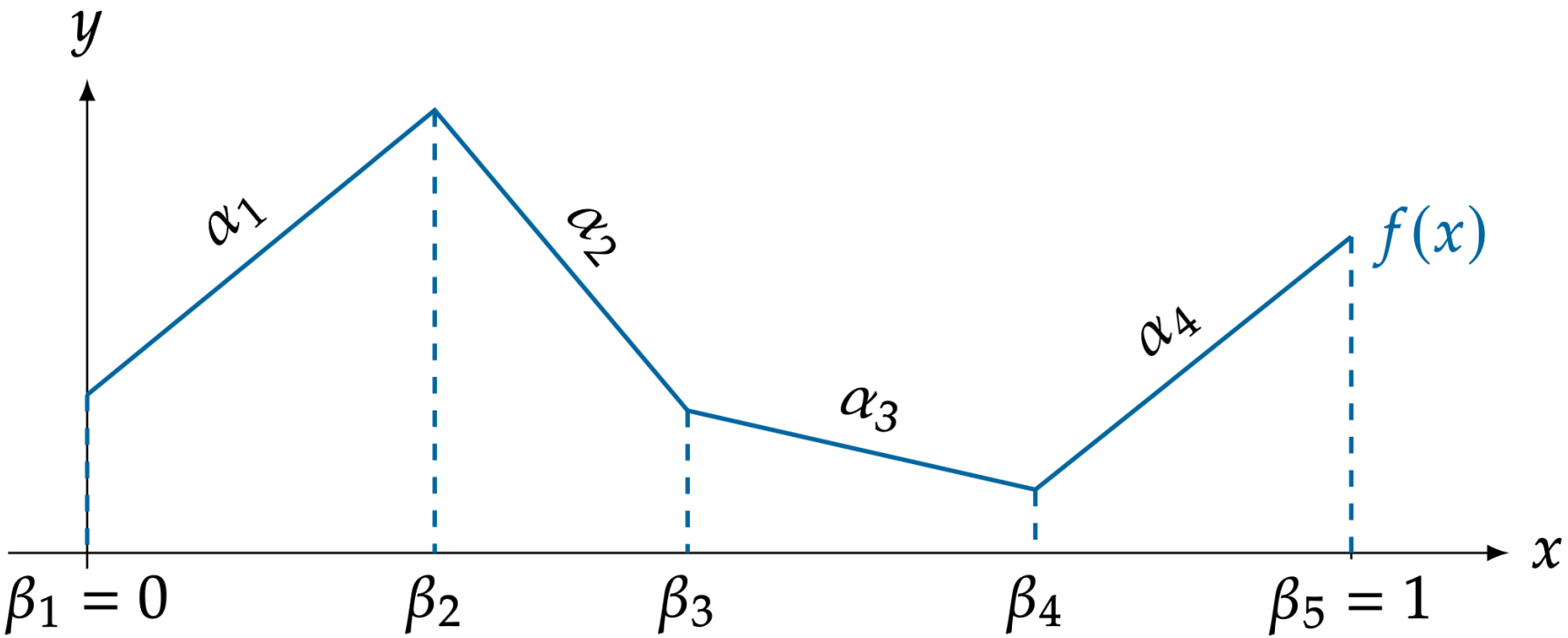
ReLU activation function



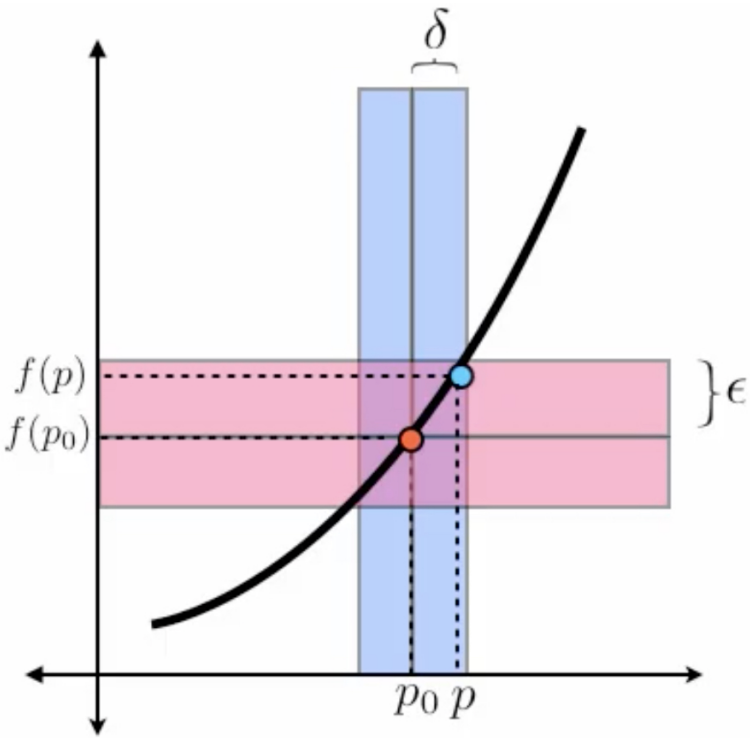
Piecewise linear functions

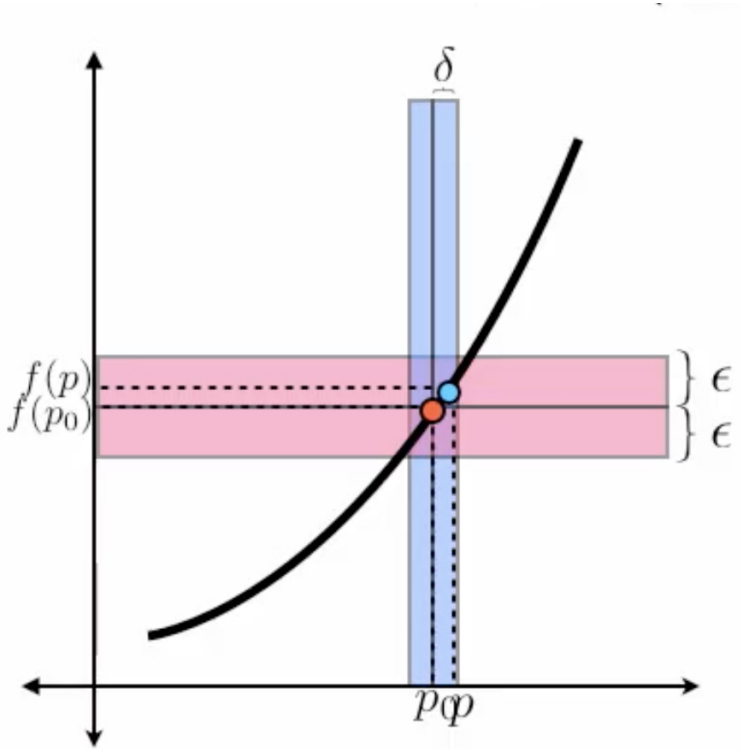


Example: Piecewise linear functions

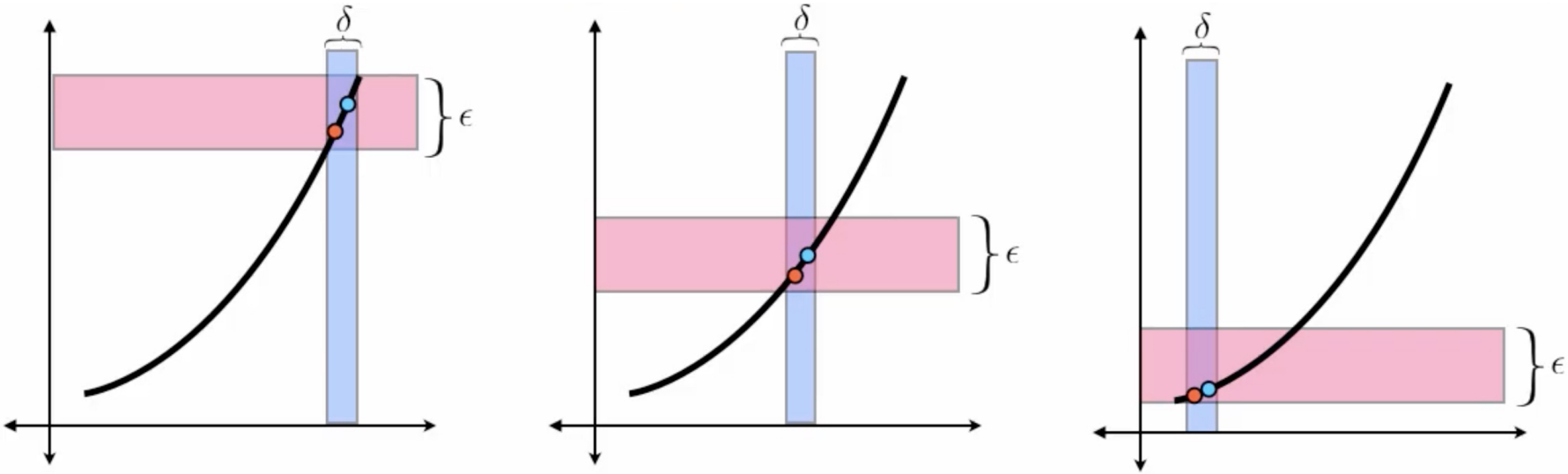


Example: Continuity

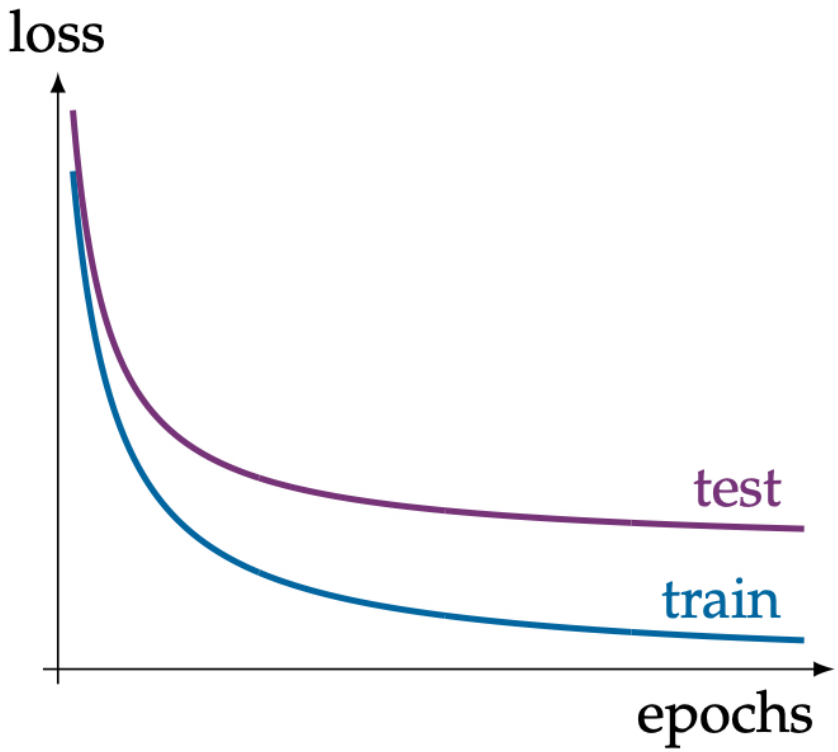


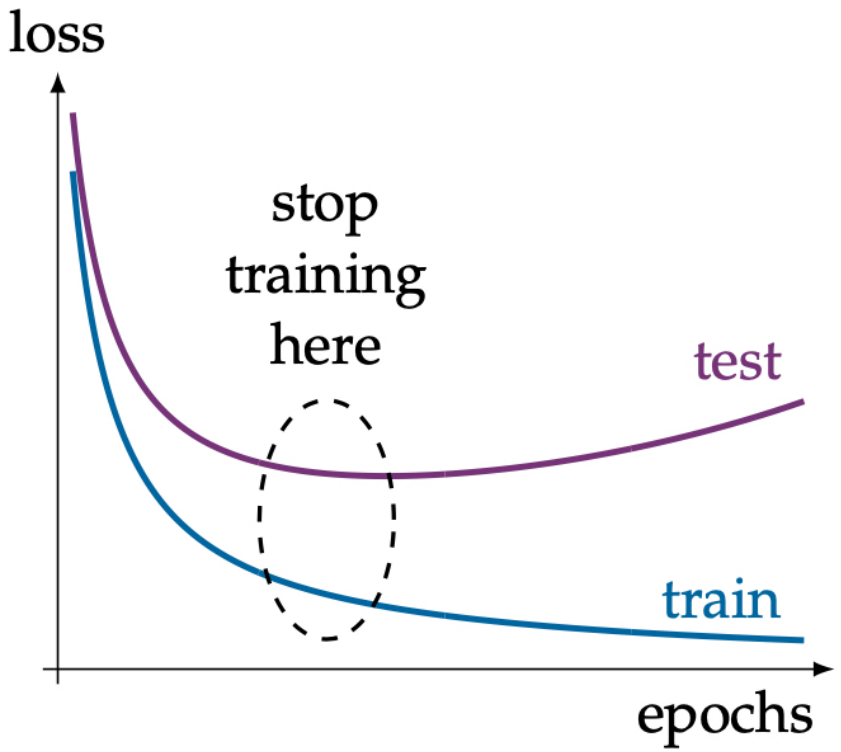


Example: Uniform continuity

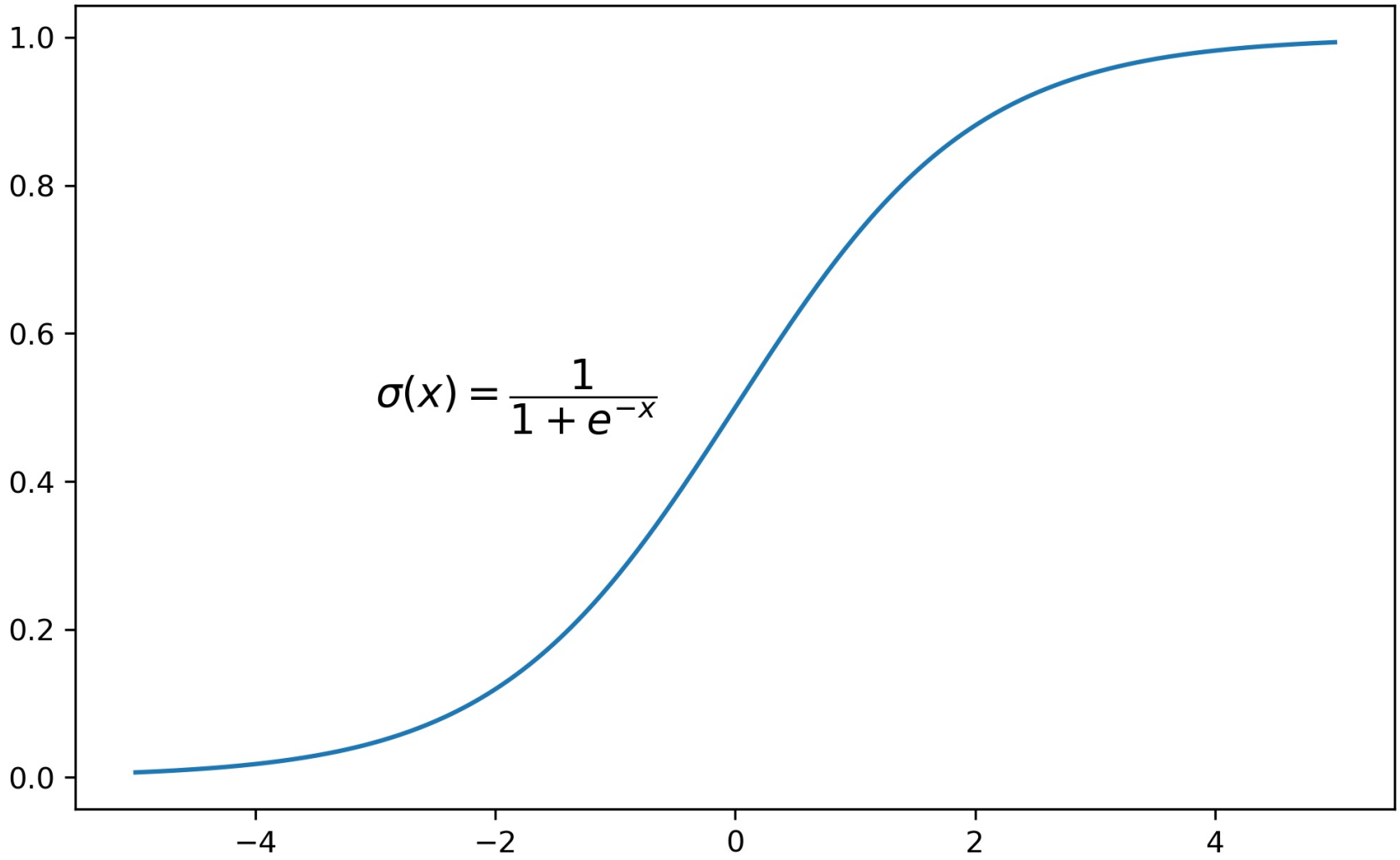


Train-test split

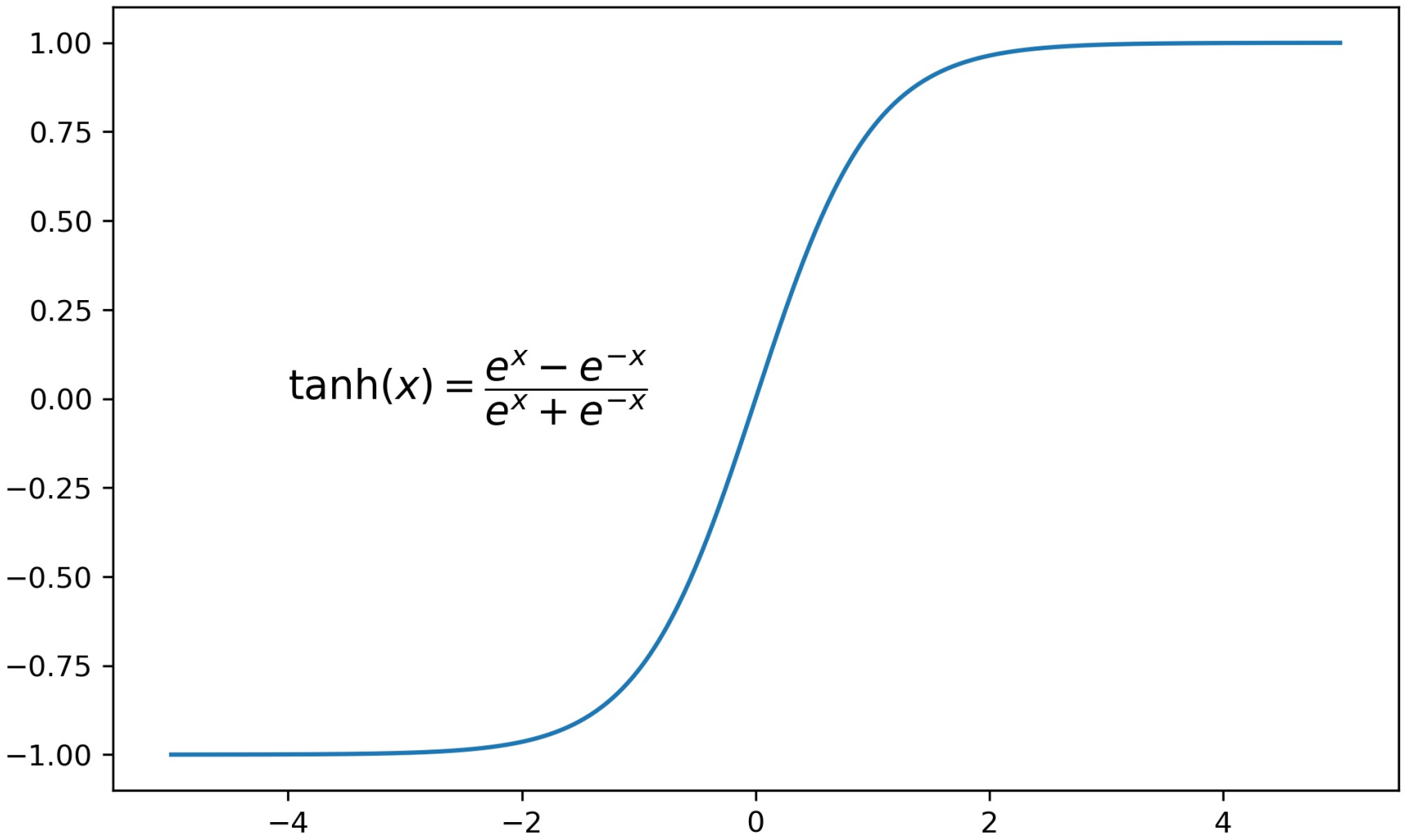




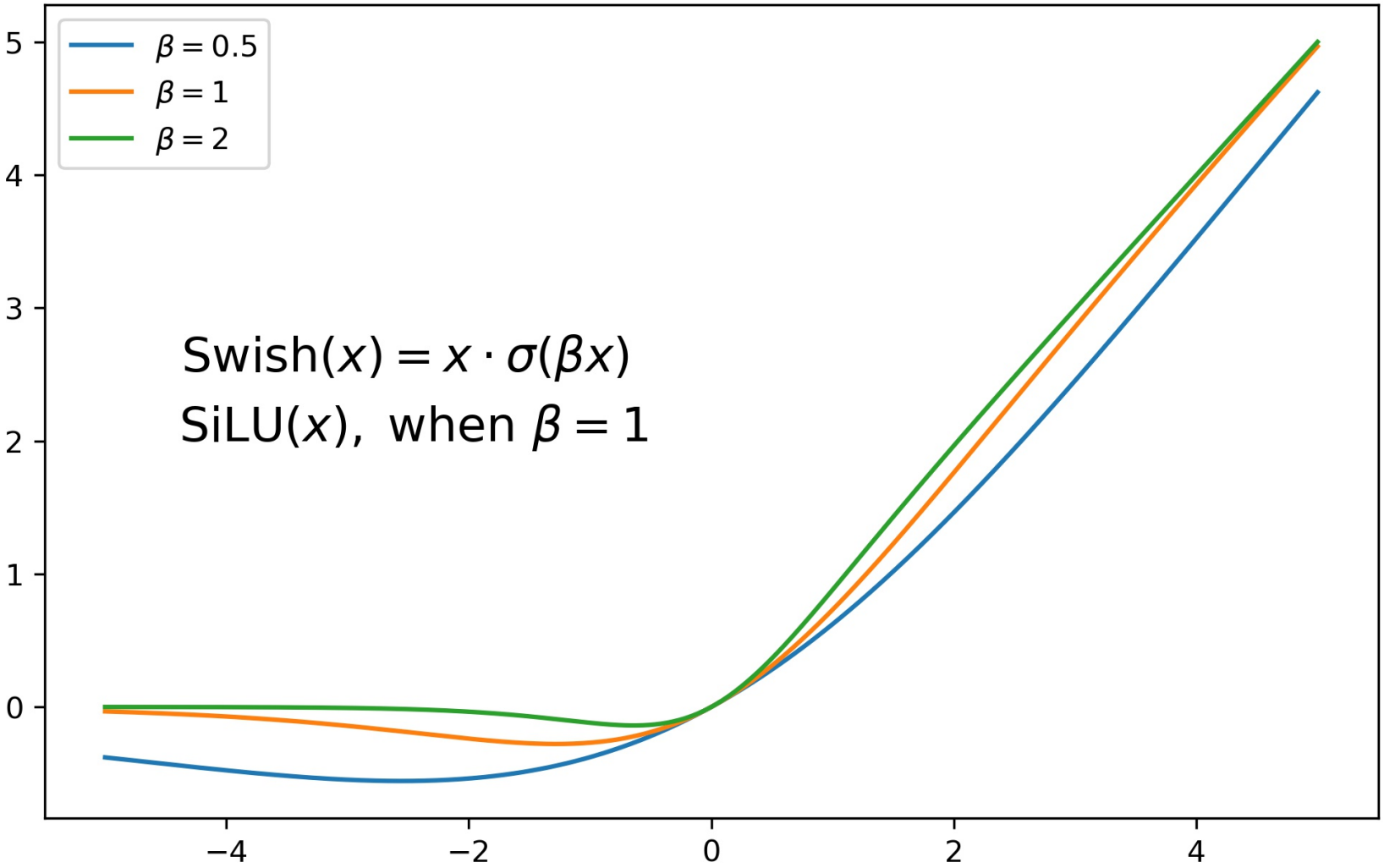
Sigmoid activation function



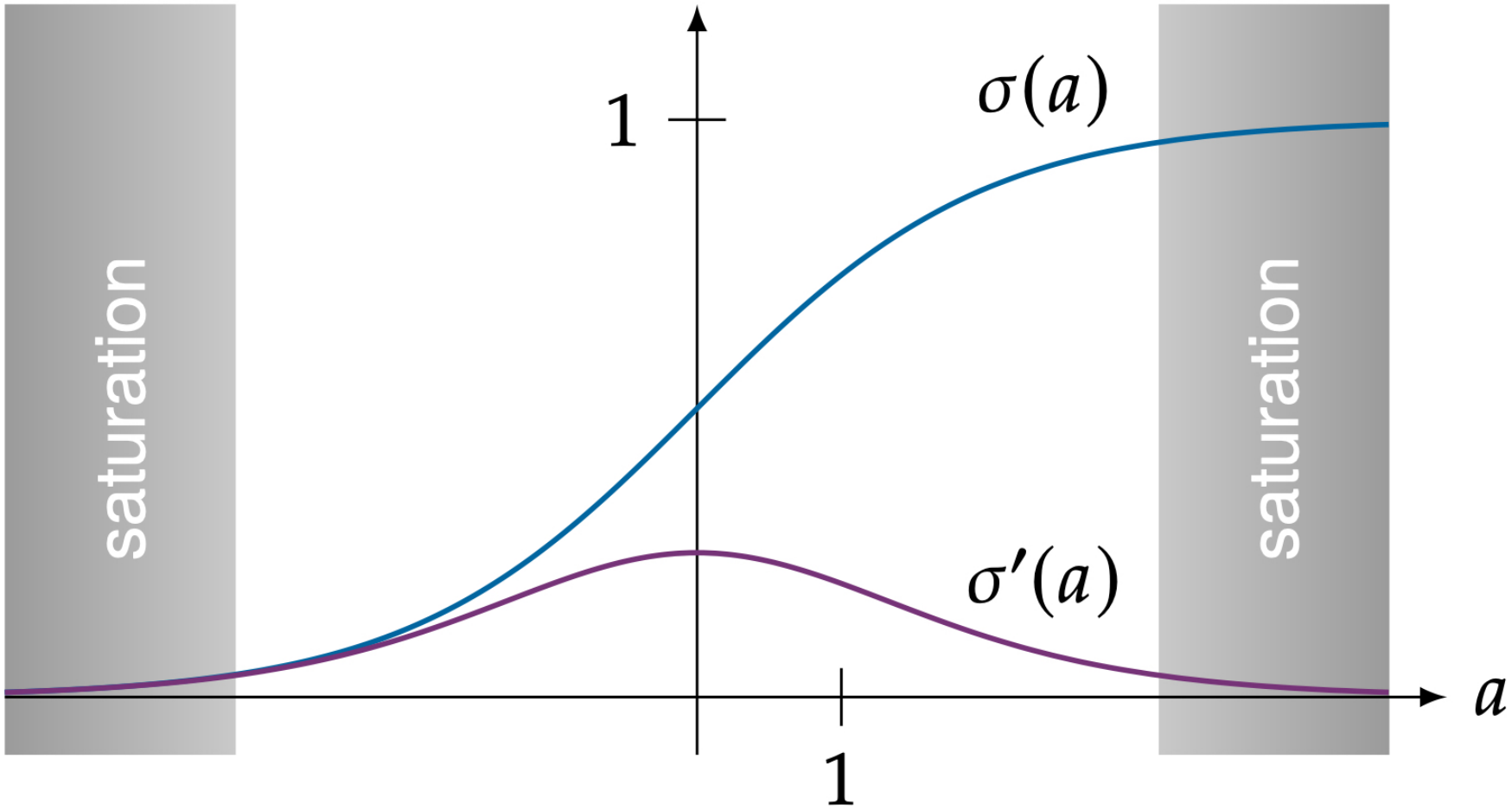
tanh activation function



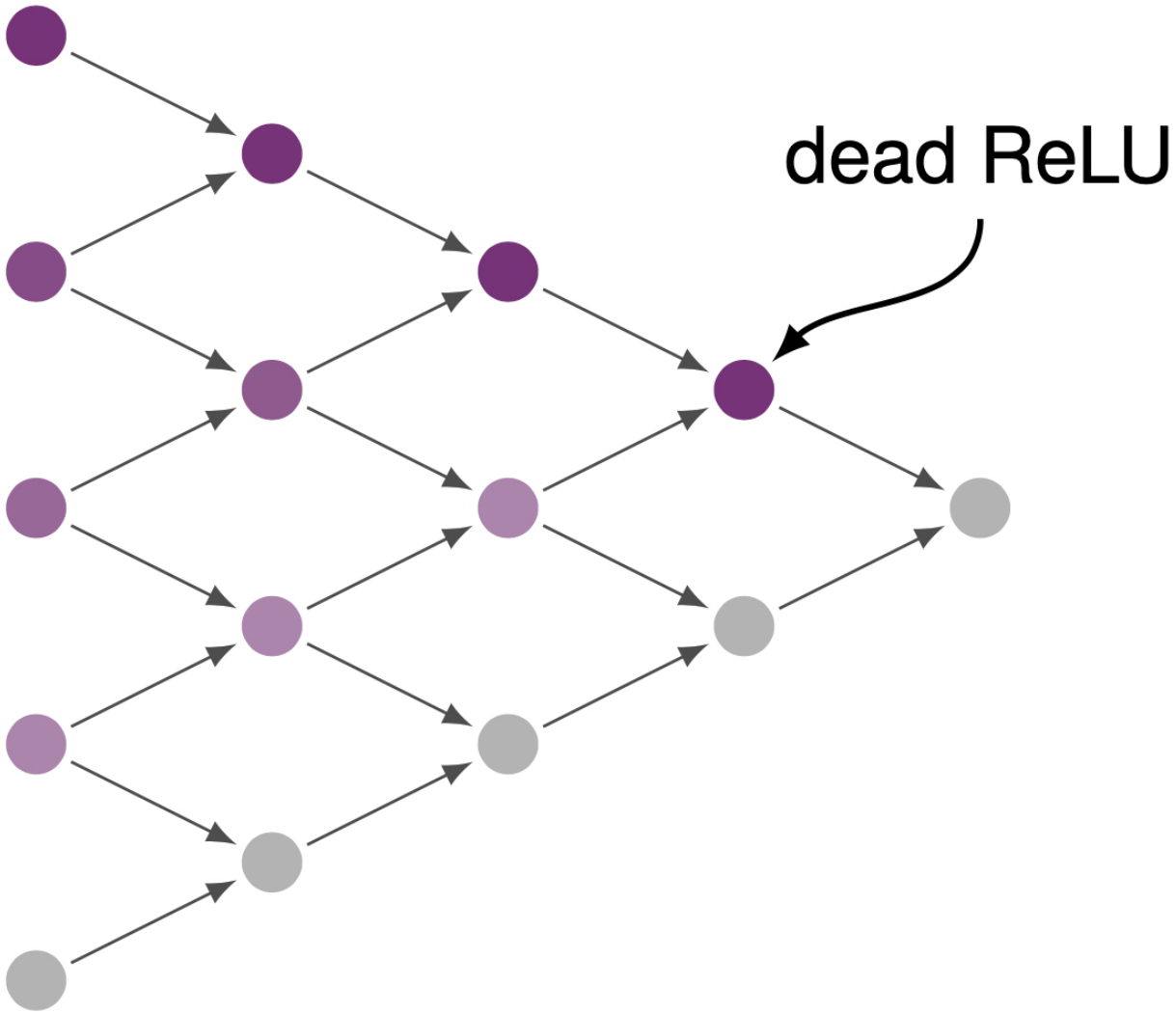
Swish / SiLU activation function



Saturation of sigmoid



Dead ReLU



Section 4

Initialization

Initialization

Consider a feedforward neural network

$$z^{(l)} = \sigma(A_l z^{(l-1)} + b_l)$$

Initialization

Consider a feedforward neural network

$$z^{(l)} = \sigma(A_l z^{(l-1)} + b_l)$$

Training requires

- ◇ forward propagation of activations
- ◇ backward propagation of gradients

Initialization

Consider a feedforward neural network

$$z^{(l)} = \sigma(A_l z^{(l-1)} + b_l)$$

Training requires

- ◇ forward propagation of activations
- ◇ backward propagation of gradients

Poor initial paramters

- ◇ being too small in magnitude $\sim$ vanishing gradient problems
- ◇ being too large in magnitude $\sim$ exploding gradient problems

Why random initialization?

Suppose all weights are initialized identically, i.e. $A_{ij} = c$

Why random initialization?

Suppose all weights are initialized identically, i.e. $A_{ij} = c$

$$\Downarrow$$

Every neuron computes the same function

Why random initialization?

Suppose all weights are initialized identically, i.e. $A_{ij} = c$

$$\Downarrow$$

Every neuron computes the same function

During gradient descent

- ◇ gradients are identical
- ◇ parameter updates remain identical

Why random initialization?

Suppose all weights are initialized identically, i.e. $A_{ij} = c$

$$\Downarrow$$

Every neuron computes the same function

During gradient descent

- ◇ gradients are identical
- ◇ parameter updates remain identical

$$\Downarrow$$

Neurons remain identical forever

Why random initialization?

Suppose all weights are initialized identically, i.e. $A_{ij} = c$

$$\Downarrow$$

Every neuron computes the same function

During gradient descent

- ◇ gradients are identical
- ◇ parameter updates remain identical

$$\Downarrow$$

Neurons remain identical forever

Random initialization breaks this symmetry.
Each neuron starts with a slightly different function

General case

Theorem: Let X be a real valued random variable and let $f : \mathbb{R} \to \mathbb{R}$ be a differentiable function then

$$Var(f(X)) \approx f'(\mathbb{E}[X])^2 \cdot Var(X)$$

assuming the variance of X is finite and f' is differentiable.

General case

Theorem: Let X be a real valued random variable and let $f : \mathbb{R} \to \mathbb{R}$ be a differentiable function then

$$Var(f(X)) \approx f'(\mathbb{E}[X])^2 \cdot Var(X)$$

assuming the variance of X is finite and f' is differentiable.

If the variance of X is small and f' is reasonably bounded then the variance of $f(X)$ will also be small

General case

Theorem: Let X be a real valued random variable and let $f : \mathbb{R} \to \mathbb{R}$ be a differentiable function then

$$Var(f(X)) \approx f'(\mathbb{E}[X])^2 \cdot Var(X)$$

assuming the variance of X is finite and f' is differentiable.

If the variance of X is small and f' is reasonably bounded then the variance of $f(X)$ will also be small

Condition for stable signal propagation

Assuming $\mathbb{E}(X_1) = \cdots = \mathbb{E}(X_n)$ and $Var(X_1) = \cdots = Var(X_n)$ we have

$$\mathbb{E}\left[\sigma'\left(\mathbb{E}(X_1)\sum_{j=1}^{n} A_{ij} + b_i\right)^2 \left(\sum_{j=1}^{n} A_{ij}^2\right)\right] = 1$$

Examples

Aims

- Choosing a probability distribution for the linear coefficients and one for the biases
- Balanced initialization, i.e. $\mathbb{E}(A_{ij}) = 0$ and $\mathbb{E}(b_i) = 0$

Examples

Aims

- ◇ Choosing a probability distribution for the linear coefficients and one for the biases
- ◇ Balanced initialization, i.e. $\mathbb{E}(A_{ij}) = 0$ and $\mathbb{E}(b_i) = 0$

Sigmoid with balanced inputs

$$\mathrm{U}[-4\sqrt{3}/n, 4\sqrt{3}/n]$$

ReLU with balanced inputs

$$\mathrm{U}[-\sqrt{6/n}, \sqrt{6/n}]$$

Examples

Aims

- ◇ Choosing a probability distribution for the linear coefficients and one for the biases
- ◇ Balanced initialization, i.e. $\mathbb{E}(A_{ij}) = 0$ and $\mathbb{E}(b_i) = 0$

Sigmoid with balanced inputs

$$\mathrm{U}[-4\sqrt{3}/n, 4\sqrt{3}/n]$$

ReLU with balanced inputs

$$\mathrm{U}[-\sqrt{6/n}, \sqrt{6/n}]$$

The signals will not explode or die out as they travel through the network at least at the start of training

Variance across layers

Assume weights are initialized as

$$A_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Variance across layers

Assume weights are initialized as

$$A_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Across layers we obtain approximately

$$Var(z^{(l)}) \approx n\sigma^2 Var(z^{(l-1)})$$

Variance across layers

Assume weights are initialized as

$$A_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Across layers we obtain approximately

$$Var(z^{(l)}) \approx n\sigma^2 Var(z^{(l-1)})$$

This leads to three regimes

Variance across layers

Assume weights are initialized as

$$A_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Across layers we obtain approximately

$$Var(z^{(l)}) \approx n\sigma^2 Var(z^{(l-1)})$$

This leads to three regimes

Remark: Here we assumed that $b_i = 0$ which is not a problem since $Var(X + c) = Var(X)$ for all c

Vanishing and exploding activations

Vanishing and exploding activations

Vanishing activations

$$n\sigma^2 < 1 \quad \sim \quad \text{Signals shrink exponentially across layers}$$

Vanishing and exploding activations

Vanishing activations

$$n\sigma^2 < 1 \quad \sim \quad \text{Signals shrink exponentially across layers}$$

Exploding activations

$$n\sigma^2 > 1 \quad \sim \quad \text{Signals grow exponentially}$$

Vanishing and exploding activations

Vanishing activations

$$n\sigma^2 < 1 \quad \sim \quad \text{Signals shrink exponentially across layers}$$

Exploding activations

$$n\sigma^2 > 1 \quad \sim \quad \text{Signals grow exponentially}$$

Both situations make training difficult

Desired initialization

Ideally we want the forward signal scale to remain stable

$$Var(z^{(l)}) \approx Var(z^{(l-1)})$$

Desired initialization

Ideally we want the forward signal scale to remain stable

$$Var(z^{(l)}) \approx Var(z^{(l-1)})$$

This requires

$$n\sigma^2 \approx 1 \quad \Rightarrow \quad \sigma^2 \approx \frac{1}{n}$$

This principle motivates modern initialization schemes

Xavier initialization

In this case

$$A_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

or

$$A_{ij} \sim U\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right]$$

where

- $n_{in} \sim$ number of inputs to the layer
- $n_{out} \sim$ number of neurons to the layer

Xavier initialization

In this case

$$A_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

or

$$A_{ij} \sim U\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right]$$

where

- $n_{in} \sim$ number of inputs to the layer
- $n_{out} \sim$ number of neurons to the layer

It balances forward signal variance and backward gradient variance

Xavier initialization

In this case

$$A_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

or

$$A_{ij} \sim U\left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right]$$

where

- $n_{in} \sim$ number of inputs to the layer
- $n_{out} \sim$ number of neurons to the layer

It balances forward signal variance and backward gradient variance

Remark: If $n_{in} \approx n_{out}$ then $2/(n_{in} + n_{out}) \approx 1/n_{in} \equiv 1/n$

He initialization

Xavier initialization assumes activations behave roughly symmetrically. However, with ReLU activations about half of the activations become zero. This effectively reduces the signal variance by roughly a factor of two.

$$A_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

Interpretation

- ReLU removes about half of the signal
- Doubling the variance compensates for this loss
- This stabilizes signal propagation in deep ReLU networks

Practical importance of initialization

Initialization strongly affects

- ◇ training stability
- ◇ convergence speed
- ◇ gradient propagation

Practical importance of initialization

Initialization strongly affects

- ◇ training stability
- ◇ convergence speed
- ◇ gradient propagation

In practice

- ◇ poor initialization can completely prevent learning
- ◇ proper initialization enables stable training

Practical importance of initialization

Initialization strongly affects

- ◇ training stability
- ◇ convergence speed
- ◇ gradient propagation

In practice

- ◇ poor initialization can completely prevent learning
- ◇ proper initialization enables stable training

Initialization	Variance	Typical activation
Xavier	$\frac{2}{n_{in}+n_{out}}$	sigmoid, tanh
He	$\frac{2}{n_{in}}$	ReLU

Section 5

Adaptive learning rate algorithms

Reminder: optimization problem

Training a NN means minimizing a loss function

$$l_{\text{total}}(w) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} l(F(x;w),y)$$

using GD

$$w^{(k+1)} = w^{(k)} - \eta \nabla l_{\text{total}}(w^{(k)})$$

where η is the learning rate

Problem with a fixed learning rate

In deep NNs

- ◇ some parameters have large gradients
- ◇ others have very small gradients

Problem with a fixed learning rate

In deep NNs

- ◇ some parameters have large gradients
- ◇ others have very small gradients

With a single learning rate

- ◇ large gradients may cause instability
- ◇ small gradients lead to slow learning

Problem with a fixed learning rate

In deep NNs

- ◇ some parameters have large gradients
- ◇ others have very small gradients

With a single learning rate

- ◇ large gradients may cause instability
- ◇ small gradients lead to slow learning

This motivates adaptive learning rate algorithms

AdaGrad (Adaptive Gradient Descent)

AdaGrad adapts the learning rate for each parameter. Let

$$g^{(k)} = \nabla l_{\text{total}}(w^{(k)})$$

AdaGrad (Adaptive Gradient Descent)

AdaGrad adapts the learning rate for each parameter. Let

$$g^{(k)} = \nabla l_{\text{total}}(w^{(k)})$$

Define accumulated squared gradients

$$G^{(k)} = \sum_{j=1}^{k} \left(g^{(k)}\right)^2$$

AdaGrad (Adaptive Gradient Descent)

AdaGrad adapts the learning rate for each parameter. Let

$$g^{(k)} = \nabla l_{\text{total}}(w^{(k)})$$

Define accumulated squared gradients

$$G^{(k)} = \sum_{j=1}^{k} \left(g^{(k)}\right)^2$$

Update rule

$$w^{(k+1)} = w^{(k)} - \frac{\eta}{\sqrt{G^{(k)} + \epsilon}} g^{(k)}$$

About AdaGrad

Intuition

- Parameters with large past gradients receive smaller updates
- Parameters with small gradients receive larger updates

About AdaGrad

Intuition

- Parameters with large past gradients receive smaller updates
- Parameters with small gradients receive larger updates

Advantages

- automatic scaling of updates
- useful for sparse features

About AdaGrad

Intuition

- ◇ Parameters with large past gradients receive smaller updates
- ◇ Parameters with small gradients receive larger updates

Advantages

- ◇ automatic scaling of updates
- ◇ useful for sparse features

Limitation

The accumulated term $G^{(k)}$ keeps increasing. Therefore the effective learning rate

$$\frac{\eta}{\sqrt{G^{(k)}}}$$

continually decreases so learning may become extremely slow

RMSProp (Root Mean Square Propagation)

RMSProp uses the same basic idea as Adagrad, but instead of accumulating the squared gradients it uses a weighted average of the historic gradients (moving average).

RMSProp (Root Mean Square Propagation)

RMSProp uses the same basic idea as Adagrad, but instead of accumulating the squared gradients it uses a weighted average of the historic gradients (moving average). Let $\alpha \in (0, 1)$

$$v^{(k)} = \alpha v^{(k-1)} + (1 - \alpha) \left(g^{(k)} \right)^2$$

Update rule

$$w^{(k+1)} = w^{(k)} - \frac{\eta}{\sqrt{v^{(k)} + \epsilon}} g^{(k)}$$

RMSProp (Root Mean Square Propagation)

RMSProp uses the same basic idea as Adagrad, but instead of accumulating the squared gradients it uses a weighted average of the historic gradients (moving average). Let $\alpha \in (0, 1)$

$$v^{(k)} = \alpha v^{(k-1)} + (1 - \alpha) \left(g^{(k)} \right)^2$$

Update rule

$$w^{(k+1)} = w^{(k)} - \frac{\eta}{\sqrt{v^{(k)} + \epsilon}} g^{(k)}$$

Remarks

- Older gradients gradually decay
- $\alpha \sim$ forgetting factor, decay rate or smoothing factor
- Suggested hyperparameter values $\sim \eta = 0.001$ and $\alpha = 0.9$

Adam (Adaptive Moment Estimation)

In addition to storing an exponentially decaying average of past square gradients $v^{(k)}$ like RMSProp, Adam also keeps a running average of past gradients $m^{(k)}$, similar to momentum.

Adam (Adaptive Moment Estimation)

In addition to storing an exponentially decaying average of past square gradients $v^{(k)}$ like RMSProp, Adam also keeps a running average of past gradients $m^{(k)}$, similar to momentum.

Let $\beta_1, \beta_2 \in (0, 1)$

First moment (mean) estimate

$$m^{(k)} = \beta_1 m^{(k-1)} + (1 - \beta_1) g^{(k)}$$

Second moment (the uncentered variance) estimate

$$v^{(k)} = \beta_2 v^{(k-1)} + (1 - \beta_2) \left(g^{(k)} \right)^2$$

Adam

After bias correction

$$w^{(k+1)} = w^{(k)} - \eta \frac{\hat{m}^{(k)}}{\sqrt{\hat{v}^{(k)}} + \epsilon}$$

where

- $\hat{m}^{(k)} = \frac{m^{(k)}}{1-\beta_1^k}$
- $\hat{v}^{(k)} = \frac{v^{(k)}}{1-\beta_2^k}$

Adam

After bias correction

$$w^{(k+1)} = w^{(k)} - \eta \frac{\hat{m}^{(k)}}{\sqrt{\hat{v}^{(k)}} + \epsilon}$$

where

- $\hat{m}^{(k)} = \frac{m^{(k)}}{1-\beta_1^k}$
- $\hat{v}^{(k)} = \frac{v^{(k)}}{1-\beta_2^k}$

Default hyperparameter values

- $\eta = 0.001$
- $\beta_1 = 0.9$ and $\beta_2 = 0.99$
- $\epsilon = 10^{-8}$

Adam

After bias correction

$$w^{(k+1)} = w^{(k)} - \eta \frac{\hat{m}^{(k)}}{\sqrt{\hat{v}^{(k)}} + \epsilon}$$

where

- $\hat{m}^{(k)} = \frac{m^{(k)}}{1-\beta_1^k}$
- $\hat{v}^{(k)} = \frac{v^{(k)}}{1-\beta_2^k}$

Default hyperparameter values

- $\eta = 0.001$
- $\beta_1 = 0.9$ and $\beta_2 = 0.99$
- $\epsilon = 10^{-8}$

Currently one of the most widely used optimizers in deep learning

Which variant to use

The following artificial landscapes model some of the problems the algorithms may encounter and show some of the strengths and weaknesses of each method.

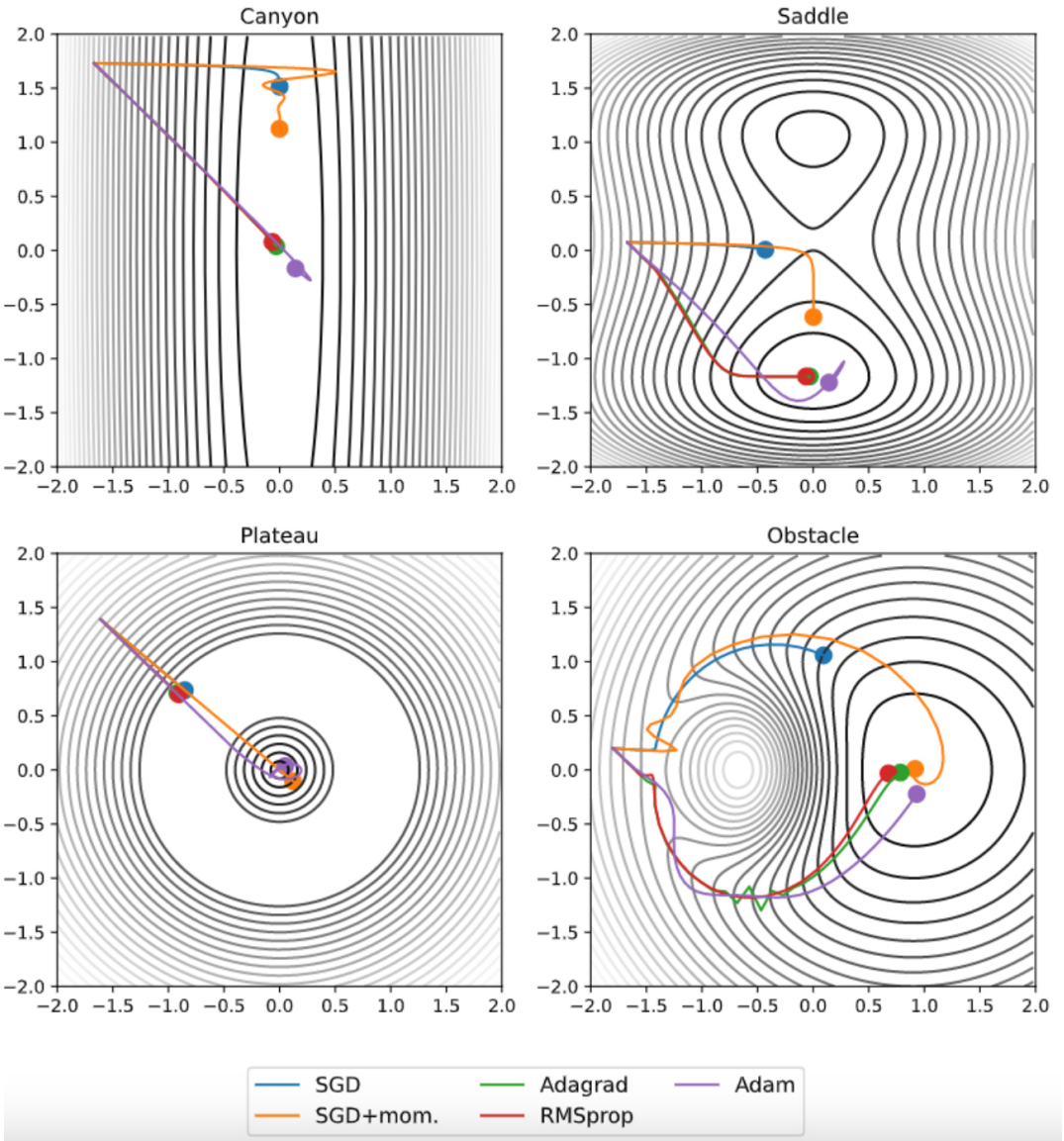
Canyon $\sim$ one parameter strongly affects the loss, the other weakly

Saddle $\sim$ prominent saddle point

Plateau $\sim$ large flat region with vanishing gradients before the minimum

Obstacle $\sim$ the algorithm must go around a barrier to reach the minimum

Which variant to use



Darker level sets indicate lower function values

Take home message

We've seen two key mechanisms stabilize training in deep networks.

Take home message

We've seen two key mechanisms stabilize training in deep networks.

Initialization

- ◇ stabilizes signal propagation
- ◇ prevents exploding or vanishing activations

Take home message

We've seen two key mechanisms stabilize training in deep networks.

Initialization

- ◇ stabilizes signal propagation
- ◇ prevents exploding or vanishing activations

Adaptive learning rate algorithms

- ◇ stabilize optimization dynamics
- ◇ adapt updates to gradient magnitudes

The End

Thank you for your attention!